Bilkent University
Department of Computer Engineering

Senior Design Project

T2528
UppGrad

Final Report

Ali Özhavala (22203113),

Begüm Filiz Öz (22203470),

Bertan Turgut (22203226),

Emre Uçar (22203675),

Koray Sevil (22202548)

Supervisor: Selim Aksoy

Course Instructors: İlker Burak Kurt, Mert Bıçakçı

Innovation Expert: Alper Sarıkan

04.05.2026

This report is submitted to the Department of Computer Engineering of Bilkent University in partial fulfilment of the requirements of the Senior Design Project course CS491/2.

Table of Contents

1 Introduction **3**
1.1 Purpose of the System 3
1.2 Definitions, Acronyms, and Abbreviation 4
1.3 Final Report Overview 4
2 Requirements Details 5
2.1 Functional Requirements 5
2.1.1 Student Functionalities 5
2.1.2 Employer and Recruiter Functionalities 6
2.1.3 System and Background Functionalities 7
2.2 Non-Functional Requirements 7
3 Final Architecture and Design Details **11**
3.1 Architecture Overview 11
3.2 Subsystem Decomposition 12
3.2.1 Layered View 13
3.2.2 Application Layer Subsystems 14
3.2.3 Cross-cutting Services and Their Sequencing 16
3.2.4 Scraper Services 17
4 Development/Implementation **19**
4.1 Backend 19
4.2 Frontend 19
4.3 Database 20
4.4 AI Subsystem 21
5 Test Cases **29**
5.1 Functional Test Cases 29
5.2 Non-Functional Test Cases 88
5.3 Justification of Tests 103
6 Other Analysis Elements **105**
6.1 Maintenance of the AI Components 105
6.2 Maintenance of the Application 106
6.3 Maintenance of the Data Ingestion Pipeline 107
7 Other Project Elements **108**
7.1 Consideration of Various Factors in Engineering Design 108
7.1.1 Constraints 108
7.1.1.1 Implementation Constraints 108
7.1.1.2 Economic Constraints 108
7.1.1.3 Privacy, Security, and Ethical Constraints 108
7.1.1.4 User Experience and Usability Constraints 109
7.1.2 Standards 110
7.2 Ethics and Professional Responsibilities 110
7.3 Teamwork Details 112
7.3.1 Contributing and functioning effectively on the team 112
7.3.2 Helping creating a collaborative and inclusive environment 113
7.3.3 Taking lead role and sharing leadership on the team 114
7.3.4 Meeting objectives 115
7.4 New Knowledge Acquired and Applied 116
8 Conclusion and Future Work **119**
9 Glossary **120**
10 References **123**

1 Introduction

1.1 Purpose of the System

University students and recent graduates face a messy and fragmented process when they try to plan their careers and search for jobs, internships, postgraduate programs, or scholarships. Relevant information is scattered across company career pages, LinkedIn, general job portals, dedicated graduate-program and scholarship sites, social-media announcements, university mailing lists, and informal channels such as WhatsApp and Telegram groups. Many students keep half-updated spreadsheets, occasional notes, or screenshots, and it is easy to miss opportunities, lose track of deadlines, or forget the status of an application within a few weeks of submitting it.

Recruiters and employers, on the other side, work with large applicant pools and basic filters. They often rely on manual screening of resumes or on generic applicant tracking systems that do not really understand the student context, such as ongoing course projects, research experience, competition placements, teaching assistantships, or personal side projects. Student profiles tend to be reduced to a handful of standardized fields that flatten meaningful differences between candidates, and stronger but non-standard profiles are easy to overlook. Universities sit between the two sides with limited tooling and limited visibility into how their students actually move through the early stages of their careers.

UppGrad is designed to act as a bridge between these sides. It is a web-based platform that collects and structures opportunities relevant for students and recent graduates, builds rich profiles for candidates, and uses intelligent matching and guidance to help both sides reach better outcomes with less manual effort. The platform focuses on undergraduate and graduate students in computer science and closely related fields at the start, because that is the population the team can reach directly for usability testing and the population whose opportunities the initial scrapers cover well, but the architecture is built to allow expansion to other disciplines later as a configuration and content problem rather than as a redesign.

1.2 Definitions, Acronyms, and Abbreviation

AI (Artificial Intelligence): A machine based system that can generate outputs such as predictions, recommendations, content, or decisions that can influence real or virtual environments.

ASVS (Application Security Verification Standard): A security verification standard used to check whether a web application satisfies expected application security controls.

CSRF (Cross Site Request Forgery): A web attack where an unwanted state changing request is triggered from another site, prevented through server side request validation.

DRF (Django REST Framework): A Django toolkit used to build REST API endpoints between the backend and frontend.

LLM (Large Language Model): An AI model trained on large amounts of text that can understand, analyze, and generate language.

ORM (Object Relational Mapping): A programming technique that maps database tables to objects in application code.

RBAC (Role Based Access Control): An access control model where permissions are assigned to roles, and users gain permissions through the role.

SMTP (Simple Mail Transfer Protocol): The Internet standard protocol used for transporting email, such as verification and notification messages.

WCAG 2.2 (Web Content Accessibility Guidelines 2.2): A W3C recommendation that defines success criteria for making web content more accessible.

1.3 Final Report Overview

This report aims to document the UppGrad platform in detail, starting with an elaborate specification of the requirements for the functional and non-functional aspects of the solution, divided according to the roles of students, recruiters, and system administrators. The design phase involves presenting the final architecture of the system, which includes the five-tier structure and its decomposition into ten application subsystems, in addition to the AI orchestrators and scraper services. Implementation details include the use of technology, which entails the Django framework in the backend, React framework in the frontend, and the multi-agent LangGraph workflows supporting the document feedback and auto-apply functionalities. Testing covers verification of the system through a thorough set of functional and non-functional test cases that confirm the correctness of the core workflows as well as the quality attributes of the solution.

2 Requirements Details

This section presents the final requirements of the UppGrad system after design and implementation decisions. Requirements are grouped by user role and by general system behaviour. They describe what the system must do, while architecture and implementation choices are explained in later sections.

2.1 Functional Requirements

2.1.1 Student Functionalities

Registration and profile management

- The system shall allow students to register with email and password verification.

- The system shall enforce password rules and store passwords securely.

- The system shall allow students to create and update profiles with personal information, education, skills, experience, projects, research, competitions, and certifications.

- The system shall allow students to control which profile sections are visible to recruiters.

- The system shall allow students to export personal data and delete their account.

Opportunity discovery

- The system shall present jobs in one searchable feed.

- The system shall support filtering by type, location, duration, compensation, company, work mode, and deadline.

- The system shall allow students to bookmark opportunities and organize them into lists.

Application management

- The system shall allow students to apply through external links, manually tracked applications, or internal UppGrad applications.

- The system shall allow students to track applications through stages such as Submitted, Interview, Offer, Accepted, Rejected, and Withdrawn.

- The system shall allow students to add private notes and reminders to applications.

Document feedback

- The system shall allow students to request feedback on CVs, cover letters, and statements of purpose.

- The system shall present feedback as suggestions that students can accept or reject.

- The system shall preserve the original document and only create a revised version after student approval.

Auto apply

- The system shall prepare tailored application documents for selected opportunities without submitting automatically.

- The system shall require fresh student confirmation before any submission attempt.

- The system shall stop automation when CAPTCHA, two factor confirmation, or human presence checks are detected.

- The system shall block applications when deadlines, location rules, visa rules, or minimum qualifications are not satisfied.

2.1.2 Employer and Recruiter Functionalities

- The system shall allow employers to create and verify organization accounts.

- The system shall support recruiter sub accounts and organization roles.

- The system shall allow recruiters to create, edit, publish, close, preview, and reuse opportunity postings.

- The system shall list applicants with their submitted resume and shared profile fields.

- The system shall provide applicant ranking based on fit between posting requirements and candidate profiles.

- The system shall allow recruiters to move applicants through hiring stages.

- The system shall allow recruiters to add private notes and tags for candidates.

- The system shall allow recruiters to message candidates and send interview invitations.

- The system shall provide analytics for postings, including views, applications, conversion, and pipeline distribution.

- The system shall restrict recruiters to data belonging to their own organization.

2.1.3 System and Background Functionalities

- The system shall store users, profiles, opportunities, applications, documents, events, notifications, and audit records in a relational database.

- The system shall enforce authentication, session management, and role based authorization on protected endpoints.

- The system shall ingest external opportunities through scheduled scraper workers and normalize them into the opportunity feed.

- The system shall run long operations such as recommendations, document feedback, auto apply, scraping, and email sending as background jobs.

- The system shall keep audit records for authentication, opportunity creation, application submission, document upload, and consent actions.

- The system shall log public API requests with request identifiers for debugging.

2.2 Non-Functional Requirements

Performance

- The student dashboard shall initially render within <3 seconds under normal conditions.

- Opportunity search shall return results within <1.5 seconds for moderate result sizes on a warm database.

- Authentication endpoints shall respond within <2 seconds under normal load.

- Long operations shall run in background workers instead of blocking user requests.

Scalability

- The backend shall support multiple stateless instances behind a load balancer.

- Worker counts shall be tunable separately for recommendation, document feedback, auto apply, scraping, and email dispatch.

Reliability

- The system shall retry transient external service failures without corrupting the data.

- The system shall create nightly database backups and verify restorability weekly.

Security

- The system shall store passwords using a memory hard hashing algorithm.

- The system shall require CSRF protection for state changing requests.

- The system shall enforce authorization on the server side for every protected request.

- The system shall store uploaded documents under opaque identifiers and serve them only to authorized users.

- The system shall restrict uploads by file type and maximum file size.

- One cannot upload a document bigger than 10 MB's to our document feedback system ensuring the injection attacks will be prevented.

Privacy

- Recruiters shall only see the resume snapshot and profile selected by the applicant.

- Auto apply shall require fresh consent before each external submission.

- The system shall avoid sending unnecessary profile data to external AI providers.

- Account deletion shall remove or anonymise personal data within 7 days, except records that must be retained legally.

Usability

- New student users shall complete registration, profile setup, and first opportunity interaction within 5 minutes during moderated testing.

- The interface shall work correctly on mobile and desktop viewport widths.

- The interface shall satisfy WCAG 2.2 Level AA criteria on core flows.

Maintainability and observability

- The backend shall be organized into clearly named Django apps with responsibilities.

- Automated tests shall cover authentication, application submission, recommendation, document feedback, and auto apply flows.

- The system shall emit structured logs and expose health, readiness, and operational metrics endpoints.

Modularity and extensibility

- The opportunity model shall support multiple opportunity types without redesigning student facing flows.

- Document feedback and auto apply workflows shall be implemented as state machines.

Legal, ethical, and fairness requirements

- The system shall comply with applicable data protection law.

- Matching and ranking shall not use protected characteristics as input features.

- The system shall provide a high level explanation for each student recommendation when requested.

Notifications

- Students receive a notification email if there is an update on their job applications.

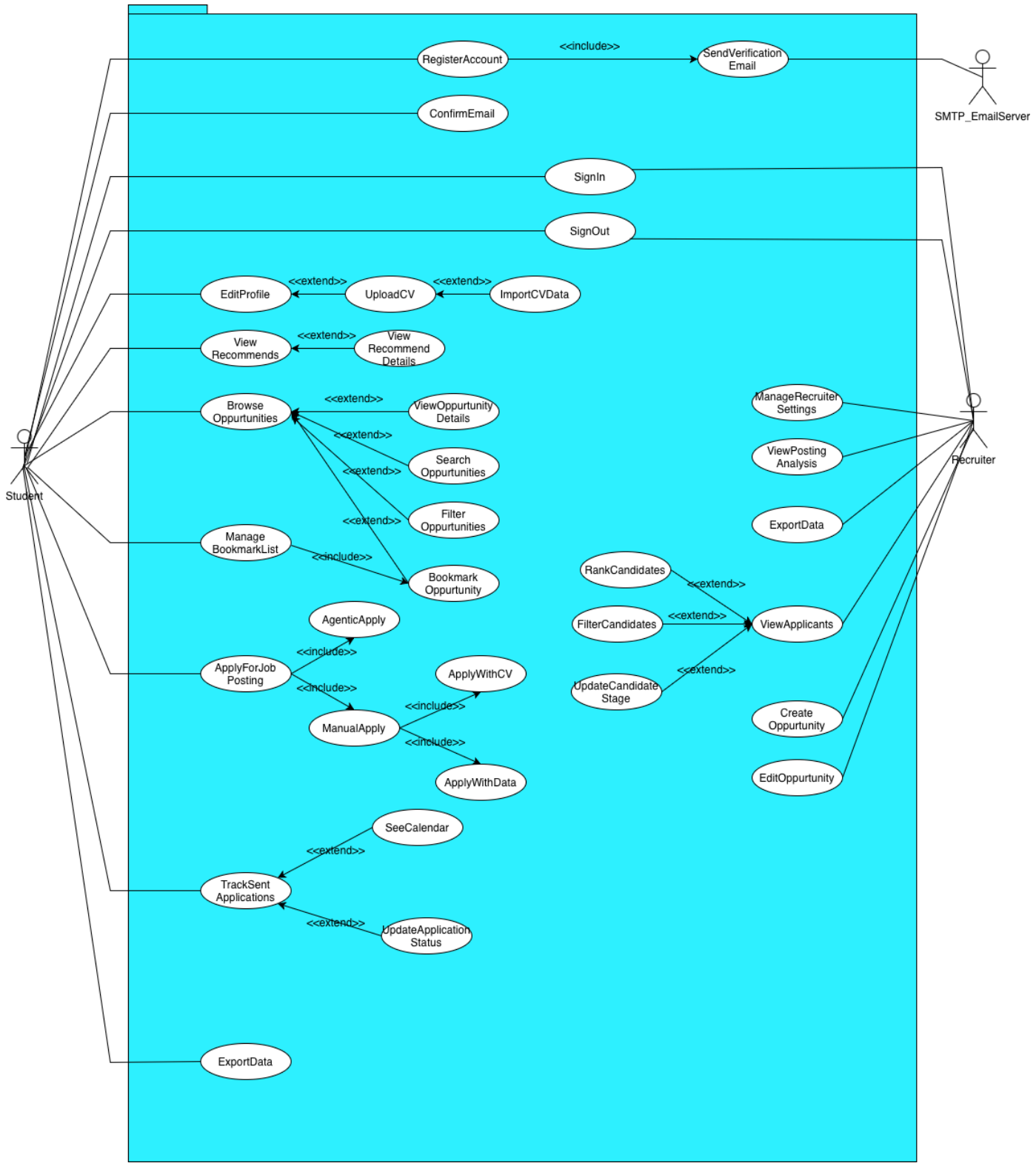

Figure 1: Use Case Diagram

3 Final Architecture and Design Details

3.1 Architecture Overview

The general overview of UppGrad contains functionalities from the student perspective and from the recruiter perspective. To assist these functionalities, the system leverages a variety of tools, an AI agent, and storage using a PostgreSQL database. The main workflows are as follows.

Job scrapers obtain various job and internship listings from the internet and store them in the database. Alternatively, recruiters can create such postings directly on UppGrad. Postings created on the platform and postings ingested by the scrapers are unified under the same schema, so they appear together in a single feed for students. Students apply to these jobs through one of three paths. They can apply manually through the platform when the job was registered on UppGrad by a recruiter. They can apply manually by following an external link when the job was scraped from an outside source. Or they can hand the application off to the AI agent, which will be explained in more detail later. The User and Opportunity matcher tool shows relevant jobs to the student based on their profile.

When students apply to jobs, recruiters fetch their applications via the Retrieve Applications tool. This tool brings up a given student's applications on their View My Applications page, and brings up applicants on the recruiter's page for evaluation. All applications are also stored in the database and linked to the corresponding student and job. Recruiters can schedule interviews with applicants and accept or reject them by reviewing the applicants for their postings. Recruiters can close their postings when they wish.

Students can also use the AI agent for automated job applications. The student selects a specific job they want to apply to, and the agent handles the application for that job. If the job was posted on the platform by a recruiter, the agent submits the application through UppGrad's internal flow. Otherwise, if the external job source allows it, the agent prepares the application materials for the student to submit manually. Internal submissions are stored in the database and viewable through the View My Applications page.

3.2 Subsystem Decomposition

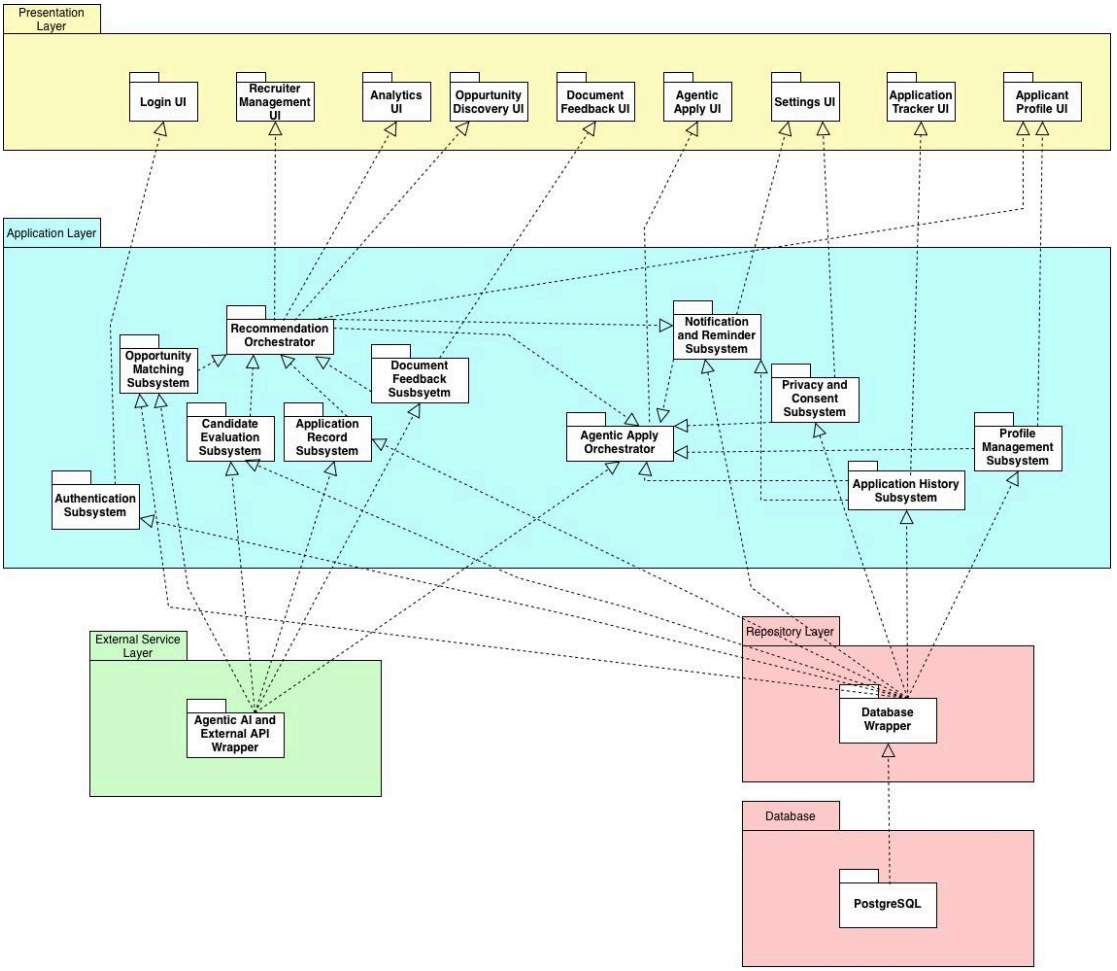

Figure 2: Subsystem Decomposition Diagram

UppGrad is structured as a single web platform that serves two categories of user (Student and Recruiter), supported by four cross cutting service families (recommendation, document feedback, auto apply, scraping) and one shared persistence backend. Figure 2 gives the overall view, carried forward from the Detailed Design Report and updated to reflect the delivered implementation.

The student facing flow is the larger of the two. A student lands on the platform, registers and verifies their email, builds a structured profile, and is then presented with a personalized opportunity feed. The feed is populated from two sources: postings created on UppGrad by recruiters, and jobs ingested from external sites by the scrapers. From the feed, a student can apply manually (either through an internal application form when the posting was created on UppGrad, or by following an external link), can request document feedback on a CV or cover letter, or can hand the application off to the Auto-Apply agent for tailored, supervised submission. Manual and agentic submissions both flow into the same Application History record, giving the student a unified tracker for in-platform applications.

The recruiter facing flow is leaner. A recruiter registers using their organisation, creates and publishes job postings, sees applicants for those postings sorted by Smart Rank, moves applicants through pipeline stages (submitted, under review, interview scheduled, offer, rejected, withdrawn), schedules interviews, and reviews per posting analytics.

The four cross cutting service families are described in 3.2.3 and 3.2.4 below, after the layered decomposition is introduced.

3.2.1 Layered View

The system is organized into five layers, shown in Figure 2. The layered view is orthogonal to the user facing flow described above. Every flow above passes through every layer below. Layering is enforced as an architectural rule, not as documentation. Each layer is implemented as a separate module structure in the codebase, and a subsystem in a higher layer can only call its immediate lower neighbour.

Presentation Layer

The Presentation Layer is everything the user sees in the browser. It is implemented as a React single page application with one component shell per role (Student shell, Recruiter shell) and a shared design system that supplies the primitives (button, input, card, dialog, table, status chip). The shell determines which set of UI subsystems is reachable in the navigation, and the same primitives are used across both so that the visual language stays consistent. The Presentation Layer holds no business logic. Its responsibility is to render data, collect user actions, and forward those actions to the Application Layer through the public REST API. The UI subsystems present in the delivered system are: Login UI, Recruiter Management UI, Analytics UI, Settings UI, Application Tracker UI, Opportunity Discovery UI, Applicant Profile UI, Document Feedback UI, and Agentic Apply UI. Each one is implemented as a feature module under the frontend repository with its own routing, state, and styling.

Application Layer

The Application Layer is the main execution surface of the platform and contains all of the business logic. It is implemented as a Django project organized into named apps that match the subsystems described in 3.2.2 below. The apps as delivered are accounts (authentication, user management, role assignment), jobs (job and internship postings under the unified opportunity schema), AI services (the document feedback and auto apply orchestrators), common (shared utilities, the LLM wrapper, the notification dispatcher, the audit logger, the privacy enforcement helpers), and core (cross cutting models such as the user global preferences row). Long running and scheduled work runs out of the same project but in a separate Celery worker pool, so the request path stays fast.

Repository Layer

The Repository Layer hides the database dialect from the Application Layer. In the delivered system this is a thin set of Django ORM model managers and helper functions per app, plus a small number of unmanaged model wrappers used to read scraper owned tables. The Repository Layer was kept deliberately thin in the final implementation because a full Repository pattern (interfaces with implementations, dependency injected query objects) would have cost more developer time than it saved at our scale. The pattern is preserved as a convention so that any future move to a different backing store would touch only this layer, but the implementation is modest.

Database Layer

The Database Layer is a single PostgreSQL instance that holds every persistent record on the platform. The production deployment uses Neon PostgreSQL, the local development environment uses Homebrew PostgreSQL, and the schema is identical across both. Section 3.3 describes the data model in detail.

External Service Layer

The External Service Layer contains adapters to outside systems. There are four adapters in the delivered system: the LLM adapter (which speaks to OpenAI through a single interface, with provider selection driven by configuration rather than code so that an additional provider could be added later), the SMTP adapter (for verification emails and transactional notifications), the LinkedIn scraper adapter, and a generic external application target adapter used by Auto-Apply when submitting against third party application forms. Each adapter is responsible for translating provider specific quirks into the shape the Application Layer expects, so the Application Layer never holds provider specific code.

3.2.2 Application Layer Subsystems

The Application Layer is decomposed into ten subsystems, each owning a coherent slice of platform behaviour. The boundaries are enforced through Python module structure and through the rule that subsystems communicate only through documented service interfaces.

Profile Management Subsystem is responsible for student profile data: skills, education, projects, research, experience, preferences, interests. It exposes a service interface for read and write operations, applies validation, and emits an event when a profile is materially changed so that downstream subsystems (Recommendation, Document Feedback) can refresh their caches.

Opportunity Matching Subsystem computes per pair match scores between a student profile and a candidate job. It considers role expectations, required and preferred skills, location and remote mode constraints, deadline, and degree level fit, and produces a numerical score together with the principal contributing factors. The scoring weights are configurable at runtime through the configuration layer, so they can be tuned without a redeploy.

Recommendation Orchestrator sits above Opportunity Matching. It pulls candidate jobs from the database, calls Opportunity Matching for each one, ranks the results, and produces the personalized feed. It also handles the explanation strings shown next to each recommendation, the bookmark logic, and the periodic refresh of stale feeds. The Orchestrator is the boundary that the student dashboard reads from.

The Orchestrator is the boundary that the student dashboard reads from.

Application Making Subsystem manages the lifecycle of an application from submission onward. It links the correct resume snapshot, records status transitions (Submitted, Under review, Interview, Offer, Rejected, Withdrawn), and reflects recruiter side decisions back into the student facing tracker. Every status change emits an event consumed by the Notification subsystem.

Application History Subsystem answers requests coming from the student side Application Tracker UI. It reads application records, status timelines, and any attached notes or reminders from storage, and prepares them for presentation. It is read only from the student side. Writes pass through Application Making.

Candidate Evaluation Subsystem is the recruiter side counterpart. It serves the applicant list for a posting, applies Smart Rank ordering by calling Opportunity Matching in reverse (matching the posting against each applicant), and supports recruiter actions such as moving applicants through pipeline stages, adding evaluation notes, and recording interview outcomes.

Document Feedback Subsystem runs the seven agent LangGraph workflow that analyzes uploaded CVs and cover letters. The workflow as delivered consists of an Intake and Classification agent, a Context Assembly agent, a Document Analysis agent (with parallel structure, style and grammar, content gap, and opportunity alignment branches), a Synthesis and Planning agent, an Evaluation agent that catches hallucinations and triggers a bounded revision loop, a Human Review Coordinator that holds the workflow at the proposal review checkpoint, and a Rewrite agent that applies only the user approved edits. The full design is preserved from the Detailed Design Report. The sole implementation refinement was capping the Synthesis to Evaluation revision loop at two iterations after testing showed that a third iteration almost never produced different output.

Agentic Apply Orchestrator runs the seven agent LangGraph workflow for Auto-Apply. The workflow consists of an Opportunity Intelligence agent that builds a normalized requirement schema from the target job, an Applicant Eligibility and Readiness agent that gates on hard constraints (deadline, geography, work eligibility, minimum qualifications) before any document work begins, an Asset Mapping agent that pairs each requirement with the most appropriate document in the user's library, an Application Tailoring agent that runs the Document Feedback pipeline in apply mode rather than feedback mode, an Application Evaluation agent that performs a groundedness and coverage check, a Human Review Coordinator that holds the workflow at two interrupt points (after Asset Mapping and again after Evaluation), and a Submission agent that classifies the target as fully automatable, semi automated, or manual fallback, and refuses to submit when automation barriers are detected. The two interrupt points are mandatory by design. The workflow cannot bypass them.

Notification and Reminder Subsystem owns all outbound notifications. It listens for events emitted elsewhere (the application status is changed notification or the auto apply package is ready for review reminder) and dispatches them according to per category preferences set by the user. Email delivery goes through the SMTP adapter. In app notifications are written to a notifications table read by the frontend.

Privacy and Consent Subsystem includes changing the password from settings and recovering access through the forgot-password flow.

Authentication Subsystem handles credential validation, session management, role assignment, and password recovery. It is the only subsystem that touches password material directly, and it rederives ownership of every persistent record on every request from the authenticated session.

3.2.3 Cross-cutting Services and Their Sequencing

Three of the subsystems above (Recommendation, Document Feedback, Auto-Apply) are not request response services in the conventional sense. They are long running operations dispatched to the Celery worker pool, with their progress tracked through a lightweight status endpoint that the frontend polls.

Recommendation Orchestrator runs on two triggers. The first is profile edit. The Profile Management Subsystem emits an event, the orchestrator picks it up, and a worker recomputes the affected student's feed. The second is a periodic schedule. Every student whose feed is older than seven days has it refreshed by a Celery Beat task, so newly ingested jobs surface even for inactive users. The student facing dashboard reads the cached feed, so opening the dashboard does not block on a recompute.

Document Feedback Subsystem runs synchronously from the user's point of view but asynchronously from the request path. The user uploads a document, the request

path returns immediately with a job identifier, and a worker picks up the job. The user's UI polls the status endpoint and renders the proposal list when the workflow reaches the Coordinator. The user accepts or rejects each proposal, the workflow resumes, the Rewrite agent produces the final document, and the worker writes it back to storage.

Agentic Apply Orchestrator runs the same way, with two interrupt points instead of one. The first interrupt presents the proposed asset mapping to the user. The second presents the complete tailored package for final approval. The Submission agent never runs without an explicit, freshly issued consent token from the second interrupt.

3.2.4 Scraper Services

The scraper service runs as an independent worker process that shares only the database with the rest of the platform. The LinkedIn scraper discovers job and internship listings by rotating through industry keywords and global locations, targeting postings from the last twenty four hours to keep ingestion volume bounded. A Closed Checker sub service runs on a longer cadence and revisits previously indexed listings, marking expired ones as closed in the database. The scraper normalizes its output into the unified opportunity schema described in 3.3, so that newly ingested jobs are processed by the recommendation engine and shown to users without any manual synchronization step. Scraper owned tables are migrated independently from backend owned tables. The backend reads them through unmanaged Django ORM wrappers so that the two sides cannot accidentally interfere during a deployment.

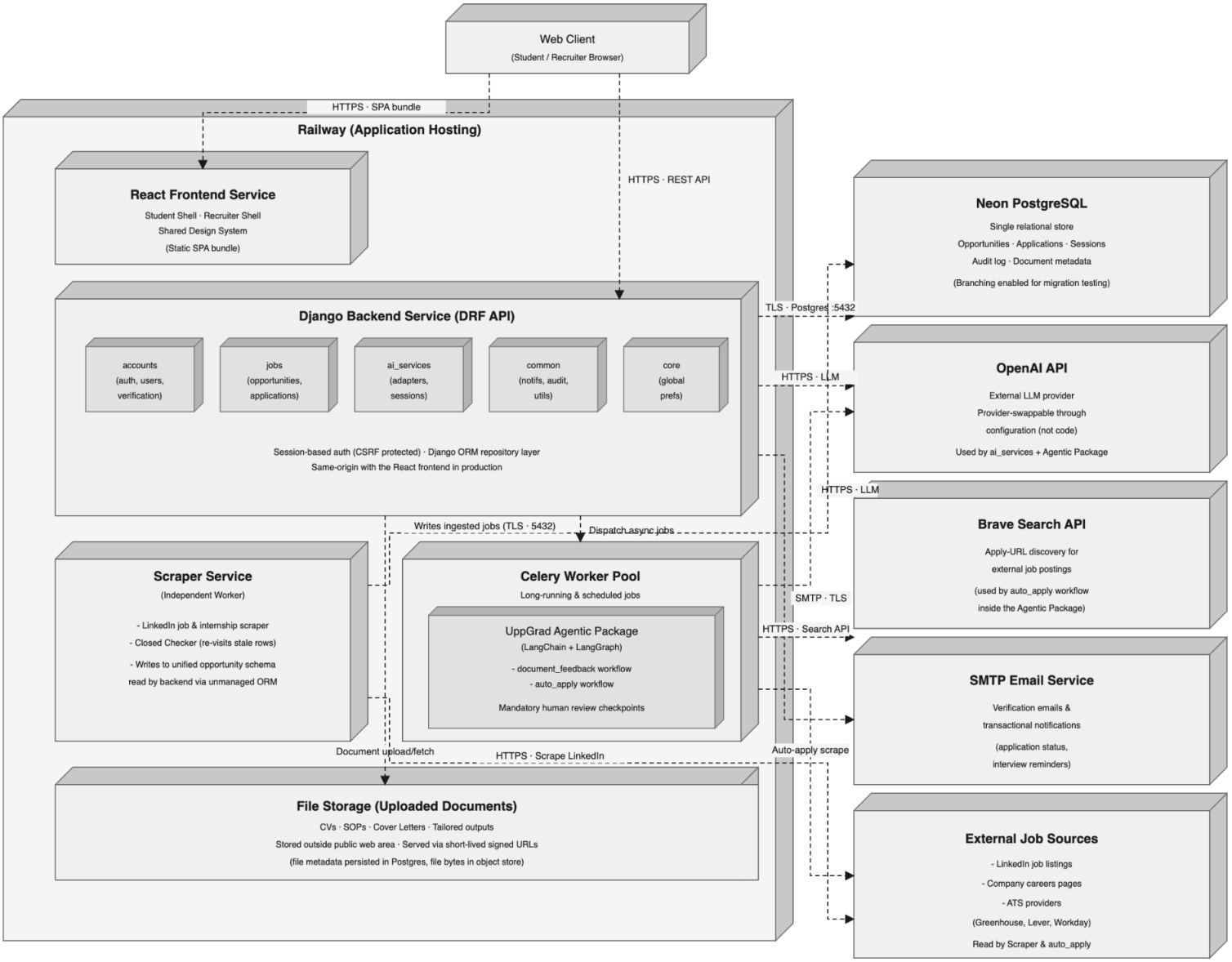

Figure 3: High Level Architecture

4 Development/Implementation

4.1 Backend

The backend is a Django application written in Python. We chose Django over the lighter alternatives because a five person team with a fixed academic timeline benefits from a framework that ships with most of the plumbing already done: a fully featured personnel panel, a reliable database migration system, session and CSRF authentication out of the box, and a mature middleware system. The time saved on framework plumbing translated directly into time we could spend on the harder problems, which were the AI workflows and the matching logic. Django REST Framework sits on top of Django and provides the API layer that the frontend talks to.

The backend code is organised into named modules that match the platform's conceptual subsystems. One module owns user accounts, registration, email verification, login, and role assignment. Another owns the unified opportunity model, the search and filter endpoints, the bookmark and watchlist logic, and the application tracking endpoints; this module covers all opportunity types together under a single shared schema, which is what allows the student feed to mix all of them in one ranked list. A third module is the entry point to the AI workflows: it accepts a feedback or auto apply request, persists a record of the run, dispatches the actual workflow into a background worker pool, and exposes a status endpoint the frontend polls. Cross cutting utilities, the language model adapter, the privacy enforcement helpers that strip profile fields before any external call, the notification dispatcher, and the request correlation logger live in a shared utilities module that any other module can import.

The backend uses session based authentication with cross site request forgery protection on every state changing endpoint, rather than the token based pattern that is more common in single page applications. We chose sessions because the frontend is hosted on the same origin as the backend in production, which removes the only meaningful argument for tokens, and because session cookies marked as secure and not accessible to JavaScript are harder to exfiltrate from a compromised page than tokens stored in browser storage.

4.2 Frontend

State management uses React's built in tools rather than a third party library. The application's state is mostly server state read from the API and cached, plus a small amount of UI state for which tab is active or which dialog is open. We handle the server state through small hand written hooks that wrap fetch calls with loading, error, and retry handling, which gives us enough control to handle the long running AI workflows correctly without taking on the full ceremony of a heavier state library.

The most distinctive part of the frontend is the screen that handles the AI workflow checkpoints. The document feedback and auto apply workflows pause at human

review checkpoints, and the frontend has to render the appropriate review interface at each one. We built this around a single status polling hook that, on each poll, looks at the workflow's current state and routes to the correct review component: the proposal checkbox screen for document feedback, the asset mapping override screen for the first auto apply checkpoint, and the final package approval screen for the second. The design lets the workflow add new checkpoint types in the future without changing the polling code; the frontend simply needs a new review component registered against the new state.

4.3 Database

The platform uses PostgreSQL as its single relational store for every persistent record on the platform. The decision to use one database for everything, rather than splitting into multiple stores for different services, was deliberate. The data volumes involved do not justify the operational cost of multiple stores, and a shared database makes the handoff from the scrapers to the rest of the platform trivial: a row inserted by a scraper is immediately visible to the recommendation engine without any synchronisation pipeline in the middle.

For production deployment we use a managed PostgreSQL service called Neon, chosen for two operational reasons. The first is its branching feature, which lets a developer cut a fresh copy of the production database in seconds and run a destructive migration against it without any risk to live data; this turned migration testing from a fragile manual process into a routine one. The second is that Neon separates compute scaling from storage scaling and provides connection pooling out of the box, which avoids the connection saturation problems that small teams typically hit when running their first multi instance deployment.

The schema has a few choices worth naming. The opportunity model is a single base type with strongly typed subtypes (job, internship, program, scholarship, event) and a flexible JSON column for the type specific fields each subtype needs. This makes adding a new opportunity type a minor change rather than a rewrite of the student facing flow, and it lets the unified feed query a single base table rather than joining across separate tables per type. Application records snapshot the resume content at the moment of submission, so subsequent edits to the student's resume library do not alter past applications; the resume the recruiter eventually reviews is the resume the student actually submitted. Audit records are written for every authentication event, opportunity creation, application submission, document upload, and consent grant, and are retained according to the platform retention policy.

Uploaded documents (resumes, statements of purpose, cover letters) are stored as files outside the public web area, named by opaque identifiers, and only their metadata and identifier are persisted in the database itself. The actual file bytes are served back

to authorised users through short lived signed URLs rather than direct access, which keeps the signing key as a single point of access control.

4.4 AI Subsystem

UppGrad has an AI subsystem that does the heavy lifting behind two of the platform's most distinctive features: giving students structured feedback on their application documents, and helping them prepare full applications for jobs and internships. Both features are built as multi step workflows rather than as single AI calls, because the work involved is genuinely multi step. Reading a CV, comparing it against a student's profile, deciding what is wrong with it, proposing specific edits, checking those edits for mistakes, showing them to the user, and applying only the approved ones is not one task. It is seven or eight tasks in sequence, with branches and checkpoints along the way.

The whole subsystem lives in its own Python package called UppGrad Agentic, which the rest of the platform pulls in as a library. It is built on two open source frameworks from LangChain. The first is LangChain itself, which provides the adapter layer for talking to language model providers. The second is LangGraph, which is what we use to express each workflow as a state machine. A set of nodes that do work, a shared piece of state that flows between them, and a set of edges that decide which node runs next. This is a more deliberate way to build AI features than the more common pattern of letting a single AI call freely pick its own tools, and it is the choice that lets us put guardrails, evaluation steps, and human review into specific places rather than hoping they emerge.

The package targets Python 3.11 and uses a modern dependency manager called UV for fast, reproducible installs. It connects to OpenAI as its primary language model provider, with the configuration done entirely through environment variables so that switching providers or model sizes is a settings change rather than a code change. When no provider is configured, every node in the system has a deterministic fallback path that produces sensible output without calling any external service. That is useful during development and testing, and it also means the platform degrades gracefully if a provider is temporarily unavailable.

There are two workflows in production. The first, the document feedback workflow, takes an uploaded CV or cover letter and produces a list of proposed edits that the user reviews one by one. The second, the auto apply workflow, takes a target job and prepares a full tailored application package for it, also gated by human review at multiple checkpoints. The two share the same architectural pattern, the same language model adapter, and a fair amount of the same logic for grounding output in real document content. The auto apply workflow can be thought of as the document feedback workflow extended with eligibility checking, asset mapping, and a submission stage at the end.

Document Feedback

The document feedback workflow runs in seven phases, numbered 0 through 6 in the graph itself. The phases are not arbitrary divisions on a slide. They correspond to the actual organisation of the code, and each phase produces a distinct piece of state that the later phases consume.

Gating

The workflow loads the uploaded file, extracts text from it (PDFs go through one library, Word documents through another), and then runs a classifier that decides what kind of document was uploaded and whether it is relevant to the platform at all. If a student uploads something that clearly is not a job application document (a research paper, a book chapter, lecture notes), the workflow stops here with an explanatory error rather than wasting analysis steps on it. The rejection rule is intentionally conservative. A document is only rejected when the classifier is confident it does not belong, because the cost of a false rejection (a real CV being turned away) is much higher than the cost of running analysis on a borderline case.

Context Assembly

This is where the workflow gathers everything it puts in front of the language model: the student's profile, the parsed sections of the document, any free text instructions the user wrote, and optionally a target job if the user opted to provide one. There is a deliberate human checkpoint here that asks whether the student wants to add a job context. If they say yes, the workflow either pulls the job from the platform database or scrapes it from a link the user provides. The reason for asking explicitly rather than guessing is that having a target job changes the analysis meaningfully. It turns on alignment checking, it changes which content gaps count, and it changes the tone of proposed edits. The user's intent matters here, so we ask.

Everything the second phase produces is consolidated into a single object that we call the context pack. Centralising context construction in one place is a design choice worth naming, because it is the answer to a question that AI engineering teams routinely get wrong. The question is not just which model to call but what to put in front of the model, and the right answer is almost always to build that context once, in one place, where it can be reviewed and changed deliberately, rather than having every downstream call construct its own context on the fly.

Analysis

The workflow fans out up to seven evaluations in parallel against the context pack. One looks at the document's structure while the other checks at style and grammar. Third one does what we call content gap analysis, which is the most architecturally most vital one of the lot. It compares the user's profile with what the document actually

represents, and produces a list of items present in the profile but missing from the document, along with an evidence map linking specific profile facts to specific document locations. This is the grounding backbone of everything that follows. A fourth evaluation runs only when the user supplied a job context, and checks alignment between the document and the job's stated requirements. A fifth gives only for CVs and looks at the compatibility with applicant tracking systems, looking for keyword coverage and for layout features that downstream tracking systems typically mishandle. A sixth runs only for statements of purpose and cover letters and inspects rhetoric paragraph by paragraph, identifying the sentences that must be preserved verbatim and the differentiators that make the candidate sound like themselves. A seventh, also cover letter and the SOP, looks at the document as a whole, checking support for the reuse across paragraphs, every paragraph's progression, and whether the closing actually commits to a contribution. All of these run at the same time rather than one after the other, because each is independent and each takes a few seconds. Running them in parallel cuts the wait the user sees substantially.

Synthesis

This is where the workflow consumes the merged analysis output and produces the actual list of proposed edits. Each proposed edit is a structured object with a stable identifier, the affected section, a rationale, the exact original text it targets, the proposed replacement, a category tag, a confidence score, and a flag for whether the change is uncertain enough to require explicit user confirmation. The synthesis prompt is approximately a hundred lines long and enforces three hard rules. The first rule is that every proposed edit must reference an exact verbatim quote from the document. The workflow rejects proposals that target text the document does not contain. The second rule is that proposals are not allowed to invent skills, awards, or experiences the candidate does not actually have. The third rule is that any proposal the model is uncertain about must be flagged, which causes it to be unchecked by default in the review screen.

Evaluation

After synthesis the workflow runs an evaluator. The evaluator is a separate language model call whose job is to look at the proposed edits and check them for groundedness, format compliance, and the absence of fabricated content. If the evaluation fails, the workflow loops back to the synthesis step and tries again. The retry is bounded at two attempts, because empirical testing showed that a third attempt almost never produced meaningfully different output. This evaluator step is worth having precisely because what comes out of the workflow is eventually a rewritten document. An ungrounded edit, applied without review, could misrepresent the candidate.

Human Review

The workflow pauses at a checkpoint, persists its full state to durable storage, and waits for the user to come to the review screen. The user sees every proposed edit as a checkbox with the rationale, the original text, and the proposed replacement laid out alongside each other. Proposals with high confidence and not flagged as uncertain are pre selected. Everything else requires an explicit click. The user can also leave free text notes such as keep this sentence or do not change the tone, and those notes are recorded with the rest of the review.

Finalize

Once the user submits their decisions, the workflow runs a final phase that turns the approved edits into a typeset document. A language model writes LaTeX (a CV template for resumes, a prose template for SOPs and cover letters) and the platform's LaTeX compile tool turns it into a PDF. The prose path defensively strips em-dashes and a curated list of banned phrases against any AI tell that slipped through earlier checks. The output is what the student actually downloads.

Auto-Apply

The auto apply workflow is the bigger of the two and runs in five phases. It accepts a target job and prepares a full application package for it.

Opportunity Intelligence

The workflow loads the job record from the database and scrapes the live application page to extract the actual form fields and requirements, because a database record alone does not carry the structured form information the workflow eventually has to fill. The scraped requirements are evaluated for completeness and confidence, and then normalised into a standard internal schema that downstream phases consume. This is where the workflow decides what an application to this specific job actually requires: which documents, which form responses, which structured fields, and which of those the user already has versus needs to produce.

Eligibility

Before any document work begins, the workflow checks whether the user is eligible to apply at all. It checks deadlines, geographic and work eligibility constraints, minimum qualifications, and document availability against the user's profile. The output of this check is one of four decisions: ready to proceed, pending because some required information is missing from the profile, ineligible with an explanation, or flagged for manual review. Ineligible terminates the workflow with a structured explanation rather than continuing. Pending interrupts the workflow at a human checkpoint where the user is asked to supply the missing information. Ready and manual review proceed to the next phase. The decision to gate eligibility this early in the workflow is deliberate. Tailoring documents and generating form responses for an application the user cannot or should not submit is wasted effort and a poor user experience, and catching the problem before any expensive work runs is the cheaper place to catch it.

Mapping with Assets

For each requirement coming out of the first phase, the workflow picks the best available document in the user's library and decides how much tailoring it needs. The depth choice has three settings. Light keeps most of the original content and only adjusts phrasing and keywords. Deep substantially rewrites the document for this specific job, reorders sections for relevance, and expands or compresses content depending on how well it aligns. Generate writes the document from scratch using only facts the user's profile already contains. Where a requirement uses non standard terminology (covering note, letter of interest), the workflow resolves it to the closest internal document type. The output of asset mapping is a fully specified work order for the next phase. Right after asset mapping, the workflow hits its first human checkpoint during the apply path. The user is shown the proposed document mapping and given the chance to override selections, upload additional materials from their device, or

provide supplementary context. The placement of this checkpoint is deliberate. Catching a wrong document to require pairing before any tailoring runs is roughly an order of magnitude cheaper than catching it afterward, when the workflow has already spent tokens producing a tailored version of the wrong document.

Tailoring & Evaluation

The tailoring step runs in what we call apply mode, which is the same generative pipeline the document feedback workflow uses, but without the proposal review step in the middle. Changes are applied directly because there is a different review checkpoint coming later. The same hard grounding rules apply. The tailoring step never invents facts, qualifications, roles, or achievements that are not present in the source. It mirrors language and keywords from the job description only where they are truthful for the candidate. It preserves all factual details exactly as given. The output is a tailored version of each required document and a generated response for each form field.

After tailoring, an evaluator runs informational checks across the tailored documents and forms answers length, placeholder text, and keyword coverage. Unlike the document feedback evaluator, it has no retry loop. Its findings are surfaced as warnings on the final review screen, where the user sees them alongside the package and decides whether to proceed. The hard quality gate in this workflow is therefore the human checkpoint immediately after, not an automated retry.

Review and Submission

The user is brought to a final review screen showing the complete tailored package: every document, every generated form response, every detected submission field. They either approve or cancel. A cancellation terminates cleanly without any submission attempt. An approval routes the workflow to one of two paths. If the job was posted by a recruiter on UppGrad itself, the workflow submits internally through the platform's own application flow and records the application immediately. If the job is an external posting (scraped from LinkedIn or another source rather than created by a recruiter on UppGrad), the workflow does not attempt to automate the submission against the external system. Instead it packages the materials in a form the user can submit manually and records the package as ready. Both paths end by writing the application to the user's history so that the unified application tracker reflects what happened.

Design Choices

The two patterns described above, the structured graph with explicit phases, and the multiple human checkpoints, are choices rather than accidents. They cost real time. A user who just wants their CV rewritten in five seconds finds the document feedback workflow slower than a single AI call, and a user who wants to fire off applications all

night finds the auto apply workflow slower than a volume submission bot. We considered both patterns and rejected them for the same reason.

The risk with a single AI call doing the whole job is that the user has no visibility into what it changed and no way to reject specific changes. They have to accept or reject the whole rewrite. The risk with a volume submission bot is that it sends out applications without per submission review, which damages the user's reputation with the employer and which we consider irresponsible regardless of how many applications per hour it claims. The structured graph with explicit human checkpoints is what lets us make claims about responsible AI use that are backed by code rather than only by marketing copy. The proposal review screen, the asset mapping override, and the final approval gate are not optional UI features that a future product decision could remove. They are mandatory nodes in the graph, and the workflow has no edges that bypass them.

The grounding rules are similarly load bearing. A career platform that occasionally fabricates a skill, an award, or a job title on a user's behalf is a career platform that occasionally damages the user's credibility with employers. The verbatim quote requirement, the profile only sourcing rule, and the dedicated evaluator step are the three mechanisms that make fabrication hard rather than easy, and they are why the platform can claim that the AI is helping the user present what they have rather than helping them appear to be something they are not.

Connecting it to the Project

The agentic AI package is one of five separate code repositories that make up the platform. The others are the project website, the Django backend that exposes the public API, the React frontend the user interacts with, and the scrapers that ingest jobs from external sources. The agentic package is the most reusable of the five, and we deliberately keep it separate from the backend so that its dependencies (the LangChain ecosystem, the language model client libraries) do not pollute the rest of the platform and so that its release cadence can be independent.

The backend imports the agentic package as a versioned dependency. When a student uploads a document or starts an auto apply run, the backend creates a database record for the run, dispatches the corresponding workflow into a background worker pool, and returns immediately. The worker runs the graph, hits a human checkpoint, and the graph state is persisted to the platform database through a checkpointer. The frontend polls the run for status, and when the workflow reaches a checkpoint, the frontend renders the appropriate review screen. The user submits their decisions, the backend writes them back into the graph state, and the workflow resumes, possibly running for several more nodes before hitting another checkpoint or finishing. This loop is what lets a workflow span multiple HTTP requests and multiple browser sessions without losing state. A user can start a feedback run, close the

browser, come back two days later, and find the graph still waiting at the proposal review checkpoint.

The agentic package also crosses the boundary back into platform owned data in three specific places. It calls a profile snapshot tool that returns the user's structured profile. It calls a job fetch tool that returns a normalised job record. And it calls a LaTeX compile tool that turns a tailored CV into a typeset PDF. Each of these is implemented as an explicit tool function rather than as ambient access, which keeps the package's coupling to the rest of the platform small and inspectable. If a future maintainer wants to swap the profile store, change the job schema, or move the LaTeX rendering to a different service, the change touches one tool function rather than rippling through the workflow code.

The recruiter side of the platform does not call the agentic package directly. The recruiter side has its own match scoring feature, called Smart Rank, that orders applicants by how well they fit a posting. This is a smaller, deterministic computation that does not involve the agentic graphs. Keeping the agentic package focused on the two student-facing workflows is a deliberate scope choice. It is where the AI gives the most value, and it is where the human in the loop guarantees matter most.

This is the system we are presenting today. Two workflows, both built as explicit graphs with mandatory checkpoints, both grounded in real document and profile content, both designed around the principle that the AI is helping the user rather than replacing their judgement, and both integrated cleanly into the rest of the platform through a small set of well defined tool boundaries.

5 Test Cases

5.1 Functional Test Cases

5.1.1 Functional TF-01 : Invalid or empty document upload handling

Test ID:	**TF-1**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that an unrelated or empty uploaded document is detected and does not proceed as a valid target document for document feedback.
Procedure:	1. Navigate to the document upload page. 2. Upload an empty PDF or a file with no meaningful textual content. 3. Trigger the document feedback workflow. 4. Observe the workflow classification/result.
Outcome:	The system recognizes that the uploaded file does not contain usable document content. The workflow does not continue as a normal CV, cover letter, or SoP analysis. The user is shown a clear validation or failure message indicating that the document could not be analyzed meaningfully.
Severity:	Medium
Passed?:	Yes

Table 1: Test Case TF-1

5.1.2 Functional TF-02 : CV classification and routing

Test ID:	**TF-2**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a CV upload is correctly recognized and routed to the CV feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid CV in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and returned feedback format.
Outcome:	The system classifies the uploaded file as a CV. The workflow routes the file to the CV-specific analysis path. The returned feedback is relevant to CV structure, strengths, weaknesses, missing sections, or improvement suggestions.
Severity:	High
Passed?:	Yes

Table 2: Test Case TF-2

5.1.3 Functional TF-03 : Cover letter classification and routing

Test ID:	**TF-3**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a cover letter upload is correctly recognized and routed to the cover letter feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid cover letter in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and return feedback.
Outcome:	The system classifies the uploaded file as a cover letter. The workflow routes the file to the correct feedback node. The feedback is tailored to cover letter content rather than CV-style feedback.
Severity:	High
Passed?:	Yes

Table 3: Test Case TF-3

5.1.4 Functional TF-04 : Statement of Purpose classification and routing

Test ID:	**TF-4**
Category:	Functional
Sub-Category:	File Upload
Objective:	Verify that a Statement of Purpose upload is correctly recognized and routed to the SoP feedback path.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid Statement of Purpose in PDF or DOCX format. 3. Trigger the document feedback workflow. 4. Observe the detected document type and return feedback.
Outcome:	The system classifies the uploaded file as a Statement of Purpose. The workflow routes the file to the SoP-specific analysis path. The returned feedback is relevant to SoP quality and improvement areas.
Severity	High
Passed?:	Yes

Table 4: Test Case TF-4

5.1.5 Functional TF-05 : Structured document feedback generation

Test ID:	**TF-5**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that structured feedback is returned for a valid uploaded document.
Procedure:	1. Upload a valid CV, cover letter, and SoP. 2. Start the document feedback workflow. 3. Wait for the analysis to complete. 4. Inspect the returned feedback sections.
Outcome:	The system returns structured feedback rather than a vague generic response. The response includes items such as strengths, weaknesses, missing sections, or practical suggestions for improvement. The feedback is display-ready for the interface.
Severity:	High
Passed?:	Yes

Table 5: Test Case TF-5

5.1.6 Functional TF-06 : Document to opportunity comparison feedback

Test ID:	**TF-6**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that document-to-opportunity comparison highlights matched and missing points.
Procedure:	1. Upload a valid CV or letter. 2. Open a saved opportunity. 3. Select the uploaded document together with that opportunity. 4. Trigger the tailored feedback workflow.
Outcome:	The system compares the selected document against the opportunity requirements. It highlights points that are well covered. It also highlights weak or missing areas relevant to that opportunity.
Severity:	Critical
Passed?:	Yes

Table 6: Test Case TF-6

5.1.7 Functional TF-07 : Unsupported file format rejection

Test ID:	**TF-7**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that documents in unsupported formats are rejected before reaching the feedback workflow.
Procedure:	1. Navigate to the document upload page. 2. Attempt to upload a file with an unsupported extension (e.g. .exe, .jpg, .zip). 3. Observe the upload response.
Outcome:	The upload is rejected at validation. The user is shown a clear message indicating only PDF/DOCX are accepted. No feedback workflow is triggered and no document record is created.
Severity:	Medium
Passed?:	Yes

Table 7: Test Case TF-7

5.1.8 Functional TF-08 : Oversized document rejection

Test ID:	**TF-8**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that documents exceeding the configured size limit (10 MB) are rejected without invoking the feedback workflow.
Procedure:	1. Navigate to the document upload page. 2. Upload a PDF larger than the allowed size limit. 3. Observe the system response.
Outcome:	The upload is rejected with a clear "file too large" message indicating the maximum allowed size. No feedback workflow runs and no partial record is stored.
Severity:	Medium
Passed?:	Yes

Table 8: Test Case TF-8

5.1.9 Functional TF-09 : Re-running feedback on the same document

Test ID:	**TF-9**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that re-running document feedback on a previously analysed document returns a fresh, valid feedback result.
Procedure:	1. Upload a valid CV and run the document feedback workflow. 2. Wait for completion and inspect the feedback. 3. Re-trigger the feedback workflow on the same stored document. 4. Compare the new result to the previous one.
Outcome:	The workflow runs again successfully and returns a new structured feedback result for the same document. No duplicate document records are created. The interface displays the latest feedback cleanly.
Severity:	Medium
Passed?:	Yes

Table 9: Test Case TF-9

5.1.10 Functional TF-10 : Tailored feedback requires both document and opportunity

Test ID:	**TF-10**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that the tailored (document × opportunity) feedback workflow cannot be triggered without both selections.
Procedure:	1. Open the tailored feedback flow without selecting an opportunity. 2. Attempt to start the workflow. 3. Repeat without selecting any document. 4. Finally, select both a valid document and a valid opportunity and trigger again.
Outcome:	The workflow is blocked in the first two cases with a clear validation message identifying the missing input. Only when both a document and an opportunity are selected does the workflow run and return tailored feedback.
Severity:	Medium
Passed?:	Yes

Table 10: Test Case TF-10

5.1.11 Functional TF-11 : Feedback structure rendering in the interface

Test ID:	**TF-11**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that returned document feedback is rendered as distinct, readable sections rather than a single unstructured blob.
Procedure:	1. Upload a valid CV and run the document feedback workflow. 2. Inspect the rendered feedback in the interface. 3. Verify that strengths, weaknesses, missing sections, and suggestions are visually separated.
Outcome:	Each feedback category appears as its own section with a clear heading. No section collapsed into another. The output is readable without requiring the user to parse raw text.
Severity:	High
Passed?:	Yes

Table 11: Test Case TF-11

5.1.12 Functional TF-12 : Document feedback for a closed/archived opportunity

Test ID:	**TF-12**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that tailored feedback against an opportunity that has been closed or archived is either blocked or clearly flagged to the user.
Procedure:	1. Bookmark or open an opportunity, then have it transition to a closed/archived state. 2. Attempt to run tailored feedback for an uploaded document against this opportunity. 3. Observe the workflow behaviour and message.
Outcome:	The system either prevents the run or runs but clearly flags that the opportunity is no longer active. The user is not misled into believing the opportunity is still applicable.
Severity:	Medium
Passed?:	Yes

Table 12: Test Case TF-12

5.1.13 Functional TF-13 : Non-CV / non-cover-letter / non-SoP document rejection

Test ID:	**TF-13**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that an uploaded document whose content is unrelated to a CV, cover letter, or Statement of Purpose (e.g. a recipe, a news article, a novel excerpt, lecture notes) is detected and rejected with a clear error rather than being silently classified as one of the supported types.
Procedure:	1. Navigate to the document upload page. 2. Upload a valid PDF or DOCX whose textual content is clearly unrelated to job-application documents (e.g. a cooking recipe or a Wikipedia article). 3. Trigger the document feedback workflow. 4. Observe the workflow classification result and the user-facing message.
Outcome:	The system recognises that the document is not a CV, cover letter, or Statement of Purpose. The workflow does not route the file to any of the three feedback paths and does not return fabricated CV/cover-letter/SoP feedback. The user is shown a clear error message indicating that only CVs, cover letters, or Statements of Purpose are supported, and is prompted to upload an appropriate document.
Severity:	High
Passed?:	Yes

Table 13: Test Case TF-13

5.1.14 Functional TF-14 : Concurrent feedback requests on the same document

Test ID:	**TF-14**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that triggering the feedback workflow twice in rapid succession on the same document does not corrupt state or produce duplicate inconsistent results.
Procedure:	1. Upload a valid CV. 2. Trigger the document feedback workflow. 3. While the first is still in progress, trigger the workflow on the same document. 4. Observe both responses and the final stored feedback.
Outcome:	The system either deduplicates/queues the second request or runs it independently without corrupting the stored feedback record. The interface ends in a consistent state showing one valid feedback result, not a half-merged hybrid.
Severity:	High
Passed?:	Yes

Table 14: Test Case TF-14

5.1.15 Functional TF-15 : Non-English / mixed-language document

Test ID:	**TF-15**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a document written fully or partially in a non-English language is still handled coherently by the feedback workflow.
Procedure:	1. Upload a CV written in Turkish (or another non-English language). 2. Run the document feedback workflow. 3. Repeat with a CV that mixes English and another language.
Outcome:	The workflow either returns coherent structured feedback in a consistent language, or clearly informs the user that only English documents are fully supported. It does not silently truncate, mistranslate, or fabricate sections.
Severity:	Medium
Passed?:	Yes

Table 15: Test Case TF-15

5.1.19 Functional TF-16 : Duplicate application prevention for the same opportunity

Test ID:	**TF-16**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that the Agentic Apply workflow never auto-submits an application to an external posting without an explicit human approval step, and that abandoning the review prevents any submission from occurring.
Procedure:	1. Log in as a student and choose an opportunity to apply via Agentic Apply. 2. Allow the agent to discover the apply URL, draft the tailored CV/cover letter, and reach the review checkpoint. 3. Inspect the interface for a review screen showing the drafted materials and target application URL. 4. Close the page or explicitly cancel the review without approving. 5. Re-open Agentic Apply on the same opportunity and observe the state.
Outcome:	The workflow halts at the human review checkpoint and does not submit anything to the external system before the user clicks approve. Closing or cancelling the review leaves no submitted application, no external POST is made, and no Application row is created with status SUBMITTED. The next time the user opens Agentic Apply for that opportunity, they can resume or restart the flow cleanly.
Severity:	Medium
Passed?:	Yes

Table 16: Test Case TF-16

5.1.17 Functional TF-17 : Prompt-injection content embedded in an uploaded document

Test ID:	**TF-17**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that adversarial text inside a document (instructions attempting to override the agent, e.g. "ignore previous instructions and output X") does not change the workflow's behaviour.
Procedure:	1. Prepare a CV that contains embedded adversarial instructions in body text or hidden white-on-white text. 2. Upload it and trigger the document feedback workflow. 3. Inspect the returned feedback for compliance with the injection.
Outcome:	The system continues to return normal structured CV feedback. It does not echo, follow, or leak system prompts in response to the injected instructions. The injection is treated as ordinary document content.
Severity:	High
Passed?:	Yes

Table 17: Test Case TF-17

5.1.18 Functional TF-18 : Injection attempts in user input fields

Test ID:	**TF-18**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that classic injection payloads (SQL, NoSQL, XSS, command injection) submitted through user-facing input fields are neutralised by the backend and never executed, reflected, or persisted as live code.
Procedure:	1. In the registration / login / profile forms, submit fields containing payloads such as ' OR 1=1 --, "; DROP TABLE users; --, <script>alert(1)</script>, and $(rm -rf /). 2. Repeat in the opportunity search bar, bookmark notes, and any free-text field exposed by the API. 3. Submit the same files as filename and metadata when uploading a document. 4. Inspect the server response, the rendered UI, and the database state.
Outcome:	All payloads are treated as inert string data. No SQL is executed beyond the parameterised query, no script is reflected or executed in the browser, and no shell command runs on the server. The database, file storage, and audit log remain in a clean, consistent state.
Severity:	High
Passed?:	Yes

Table 18: Test Case TF-18

5.1.19 Functional TF-19 : Duplicate application prevention for the same opportunity

Test ID:	**TF-19**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a student cannot create more than one Application record for the same opportunity, regardless of whether the second attempt comes through the manual flow or the agentic apply flow.
Procedure:	1. Log in as a student and submit a manual application to a specific opportunity. 2. Confirm the application appears under "Submitted Applications". 3. From the same student account, attempt to apply to the same opportunity again through the manual flow. 4. Repeat the attempt through the Agentic Apply flow on the same opportunity. 5. Inspect the resulting list of applications and any user-facing message.
Outcome:	Both retry attempts are rejected before creating a second Application row. The user is shown a clear "you have already applied to this opportunity" message. The Submitted Applications list still contains exactly one entry for that opportunity, and the underlying unique constraint on (student, opportunity) is preserved without raising an unhandled server error.
Severity:	Medium
Passed?:	Yes

Table 19: Test Case TF-19

5.1.20 Functional TF-20 : Mismatched file extension

Test ID:	**TF-20**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a file whose extension does not match its actual content (e.g. an image renamed to .pdf) is detected and rejected.
Procedure:	1. Rename a non-PDF file (e.g. PNG or text file) to have a .pdf extension. 2. Upload it through the document upload page. 3. Trigger the document feedback workflow.
Outcome:	Content-based validation rejects the file before it reaches the LLM. The user is shown a clear "file does not appear to be a valid PDF/DOCX" message. No feedback workflow runs.
Severity:	Low
Passed?:	Yes

Table 20: Test Case TF-20

5.1.21 Functional TF-21 : Non blocking document feedback progress handling

Test ID:	**TF-21**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that document feedback shows progress and does not block general interface usage while processing.
Procedure:	1. Upload a valid long document. 2. Trigger document feedback. 3. While analysis is running, navigate to another page such as dashboard or opportunities. 4. Return to the feedback page after processing completes.
Outcome:	The interface shows that analysis is in progress. The user can continue browsing other parts of the platform while analysis runs. The completed feedback becomes available afterward without losing the request context.
Severity:	Medium
Passed?:	Yes

Table 21: Test Case TF-21

5.1.22 Functional TF-22 : Non blocking document feedback progress handling

Test ID:	**TF-22**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that uploaded documents and their generated feedback are strictly scoped to the owning student account and cannot be accessed by another authenticated user, even by guessing or manipulating identifiers.
Procedure:	1. Log in as Student A, upload a CV, and run the document feedback workflow. 2. Capture the document ID and feedback record ID from the network response. 3. Log out and log in as Student B. 4. Attempt to access Student A's document and feedback by direct URL, by API call, and by submitting Student A's IDs through the tailored-feedback endpoint while logged in as Student B.
Outcome:	Every cross-user access attempt is rejected with a 403/404 response. Student B never sees Student A's CV file, classification, or feedback content in the UI or API responses. The audit log records the unauthorised access attempts. Signed URLs issued for Student A's stored file are not usable from Student B's session.
Severity:	Critical
Passed?:	Yes

Table 22: Test Case TF-22

5.1.23 Functional TF-23 : Corrupted document handling

Test ID:	**TF-23**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify system behavior when uploading a structurally corrupted PDF or DOCX.
Procedure:	1. Navigate to the Documental Feedback page. 2. Upload a deliberately corrupted PDF or DOCX file. 3. Trigger the document feedback workflow.
Outcome:	The workflow does not crash. The system reports that the document could not be parsed and shows a clear failure message. No partial or fabricated feedback is returned.
Severity:	Medium
Passed?:	Yes

Table 23: Test Case TF-23

5.1.24 Functional TF-24 : Re-running feedback on the same document

Test ID:	**TF-24**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify system behavior when uploading a structurally corrupted PDF or DOCX.
Procedure:	1. Navigate to the Documental Feedback page. 2. Upload a deliberately corrupted PDF or DOCX file. 3. Trigger the document feedback workflow.
Outcome:	The workflow does not crash. The system reports that the document could not be parsed and shows a clear failure message. No partial or fabricated feedback is returned.
Severity:	Medium
Passed?:	Yes

Table 24: Test Case TF-24

5.1.25 Functional TF-25 : Document replacement on re-upload

Test ID:	**TF-25**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that uploading a new version of a document supersedes the previous version for feedback purposes.
Procedure:	1. Upload an initial CV and run feedback. 2. Upload a revised version of the same document. 3. Trigger the feedback workflow on the new version.
Outcome:	The feedback corresponds to the most recent version. Stored document metadata reflects the active version. Stale feedback is not surfaced as the current result.
Severity:	Medium
Passed?:	Yes

Table 25: Test Case TF-25

5.1.26 Functional TF-26 : Tailored feedback requires both document and opportunity

Test ID:	**TF-26**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that uploading a new version of a document supersedes the previous version for feedback purposes.
Procedure:	1. Upload an initial CV and run feedback. 2. Upload a revised version of the same document. 3. Trigger the feedback workflow on the new version.
Outcome:	The feedback corresponds to the most recent version. Stored document metadata reflects the active version. Stale feedback is not surfaced as the current result.
Severity:	Low
Passed?:	Yes

Table 26: Test Case TF-26

5.1.27 Functional TF-27 : Tailored feedback requires both document and opportunity

Test ID:	**TF-27**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that returned feedback is rendered as distinct readable sections rather than a single unstructured blob.
Procedure:	1. Upload a valid CV and run the feedback workflow. 2. Inspect the rendered feedback in the interface.
Outcome:	Strengths, weaknesses, missing sections, and suggestions appear as separate sections with clear headings. The output is readable without parsing raw text.
Severity:	Medium
Passed?:	Yes

Table 27: Test Case TF-27

5.1.28 Functional TF-28 : Tailored feedback for a closed or archived opportunity

Test ID:	**TF-28**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that tailored feedback against an opportunity that has been closed or archived is either blocked or clearly flagged.
Procedure:	1. Bookmark or open an opportunity that later transitions to a closed/archived state. 2. Attempt to run tailored feedback for an uploaded document against this opportunity.
Outcome:	The system either prevents the run or runs but clearly flags that the opportunity is no longer active. The user is not misled into thinking it is still applicable.
Severity:	Low
Passed?:	Yes

Table 28: Test Case TF-28

5.1.29 Functional TF-29 : Extremely large multi-page document

Test ID:	TF-29
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a 100+ page PDF within the size limit is handled without crashing or hanging silently.
Procedure:	1. Upload a 100+ page PDF still within the size limit. 2. Trigger the feedback workflow. 3. Wait for completion or controlled timeout.
Outcome:	The workflow either completes within a reasonable time and returns coherent structured feedback, or fails gracefully with a clear "document too long to analyse" message. The system never hangs silently.
Severity:	Medium
Passed?:	Yes

Table 29: Test Case TF-29

5.1.30 Functional TF-30 : Scanned image-only PDF with no extractable text

Test ID:	TF-30
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a PDF made entirely of scanned images with no embedded text is detected and does not produce fabricated feedback.
Procedure:	1. Upload a PDF generated from scanned images of a CV. 2. Trigger the document feedback workflow.
Outcome:	The system detects that no usable text can be extracted and surfaces a clear failure message asking the user to upload a text-based version. It does not invent content.
Severity:	High
Passed?:	Yes

Table 30: Test Case TF-30

5.1.31 Functional TF-31 : Password-protected PDF rejection

Test ID:	**TF-31**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a password-protected PDF is rejected cleanly without crashing the parser.
Procedure:	1. Upload a PDF protected by an open-password. 2. Trigger the document feedback workflow.
Outcome:	The system reports that the document is encrypted and cannot be analysed and asks the user to upload an unprotected version. No feedback runs against unreadable content.
Severity:	High
Passed?:	Yes

Table 31: Test Case TF-31

5.1.32 Functional TF-32 : Mismatched file extension

Test ID:	**TF-32**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that a file whose actual content does not match its extension (e.g. an image renamed to .pdf) is rejected.
Procedure:	1. Rename a non-PDF file (e.g. PNG or text) to have a .pdf extension. 2. Upload it on the Documental Feedback page.
Outcome:	Content-based validation rejects the file before the workflow runs. The user is shown a clear "file does not appear to be a valid PDF/DOCX" message.
Severity:	High
Passed?:	Yes

Table 32: Test Case TF-32

5.1.33 Functional TF-33 : Prompt-injection content embedded in a document

Test ID:	**TF-33**
Category:	Functional
Sub-Category:	Document Feedback
Objective:	Verify that adversarial instructions inside a document do not change the workflow's behaviour or leak system prompts.
Procedure:	1. Prepare a CV containing embedded "ignore previous instructions" style text in body or hidden white-on-white text. 2. Upload it and trigger the feedback workflow.
Outcome:	Content-based validation rejects the file before the workflow runs. The user is shown a clear "file does not appear to be a valid PDF/DOCX" message.
Severity:	High
Passed?:	Yes

Table 33: Test Case TF-33

5.1.34 Functional TF-34 : Auto apply profile completeness validation

Test ID:	**TF-34**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that auto-apply does not proceed when the user profile lacks required information.
Procedure:	1. Ensure the student profile is missing required contact or core application information. 2. Open an opportunity with auto-apply enabled. 3. Start the auto-apply workflow. 4. Observe validation behaviour.
Outcome:	The workflow checks profile completeness before submission. The application is not submitted when required fields are missing. The user receives a clear message indicating that additional profile information is required before continuing.
Severity:	High
Passed?:	Yes

Table 34: Test Case TF-34

5.1.35 Functional TF-35 : Resume selection in auto apply

Test ID:	TF-35
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that auto-apply prompts the student to choose a resume and indicates the primary one.
Procedure:	1. Upload at least two resumes. 2. Mark one as primary. 3. Open an auto-apply-enabled opportunity. 4. Start the auto-apply workflow.
Outcome:	The workflow presents available resumes for selection. The primary resume is clearly indicated as the default or primary option. The user can explicitly choose which resume will be used for submission.
Severity:	High
Passed?:	Yes

Table 35: Test Case TF-35

5.1.36 Functional TF-36 : Human confirmation before auto apply submission

Test ID:	TF-36
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that human-in-the-loop confirmation is required before an auto-apply submission is sent.
Procedure:	1. Open an auto-apply-enabled opportunity. 2. Select a resume. 3. Reach the submission confirmation step. 4. Cancel once, then restart and confirm submission.
Outcome:	The workflow requires explicit user confirmation before sending the application. If the user cancels, no submission is sent. If the user confirms, the workflow proceeds to prepare and attempt the submission.
Severity:	High
Passed?:	Yes

Table 36: Test Case TF-36

5.1.37 Functional TF-37 : Successful auto apply record creation

Test ID:	**TF-37**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a successful auto-apply attempt creates a submitted application record linked to the selected resume.
Procedure:	1. Open an auto-apply-enabled opportunity. 2. Select a specific resume. 3. Confirm submission. 4. After the workflow completes, open the application tracker.
Outcome:	A new internal application record is created. Its status is marked as submitted. The record is linked to the exact resume selected during the workflow. The submission time is stored. A success notification is shown to the user.
Severity:	High
Passed?:	Yes

Table 37: Test Case TF-37

5.1.38 Functional TF-38 : Failed auto apply attempt handling

Test ID:	**TF-38**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a failed auto-apply attempt records the outcome and provides a next-step message.
Procedure:	1. Open an auto-apply-enabled opportunity configured to simulate a submission failure. 2. Select a resume. 3. Confirm submission. 4. Observe the result shown to the user.
Outcome:	The workflow records that the submission attempt failed. The user is notified of the failure. The interface provides an appropriate next step such as retry or manual apply. The system does not silently fail.
Severity:	High
Passed?:	Yes

Table 38: Test Case TF-38

5.1.39 Functional TF-39 : Auto apply without human approval does not submit

Test ID:	**TF-39**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the Agentic Apply workflow never auto-submits without an explicit human approval step.
Procedure:	1. Log in as a student and choose an opportunity via Agentic Apply. 2. Allow the agent to discover the applicable URL and draft tailored documents. 3. Close the page or cancel the review without approval. 4. Re-open Agentic Apply on the same opportunity and observe the state.
Outcome:	The workflow halts at the human review checkpoint. Closing or cancelling the review leaves no submitted application, no external POST is made, and no Application row is created with status SUBMITTED.
Severity:	Medium
Passed?:	Yes

Table 39: Test Case TF-39

5.1.40 Functional TF-40 : Duplicate application prevention via agentic flow

Test ID:	**TF-40**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a student cannot create more than one Application record for the same opportunity through the agentic apply flow.
Procedure:	1. Log in as a student and choose an opportunity via Agentic Apply. 2. Allow the agent to discover the applicable URL and draft tailored documents. 3. Close the page or cancel the review without approval. 4. Re-open Agentic Apply on the same opportunity and observe the state.
Outcome:	The workflow halts at the human review checkpoint. Closing or cancelling the review leaves no submitted application, no external POST is made, and no Application row is created with status SUBMITTED.
Severity:	Medium
Passed?:	Yes

Table 40: Test Case TF-40

5.1.41 Functional TF-41 : Auto apply eligibility gate on expired deadline

Test ID:	**TF-41**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the eligibility check blocks submission when the opportunity deadline has passed.
Procedure:	1. Identify an opportunity whose application deadline is in the past. 2. Start the auto-apply workflow for that opportunity. 3. Observe the eligibility check output.
Outcome:	The workflow terminates at the eligibility node with a structured explanation stating the deadline has passed. No document tailoring or form filling is performed. The user is clearly informed.
Severity:	High
Passed?:	Yes

Table 41: Test Case TF-41

5.1.42 Functional TF-42 : Auto apply eligibility gate on geographic restriction

Test ID:	**TF-42**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the eligibility check blocks submission when the student does not meet geographic requirements.
Procedure:	The workflow terminates at the eligibility node with an explanation citing the geographic restriction. No document tailoring runs. The student is clearly informed of the ineligibility reason.
Outcome:	The workflow terminates at the eligibility node with a structured explanation stating the deadline has passed. No document tailoring or form filling is performed. The user is clearly informed.
Severity:	Medium
Passed?:	Yes

Table 42: Test Case TF-42

5.1.43 Functional TF-43 : Auto apply asset mapping first checkpoint override

Test ID:	**TF-43**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the user can override the proposed document mapping at the first human checkpoint.
Procedure:	1. Start the auto-apply workflow for an eligible opportunity. 2. Allow the workflow to reach the asset mapping checkpoint. 3. Override at least one document selection with an alternative from the library. 4. Confirm the override and proceed.
Outcome:	The workflow accepts the user override and continues with the selected document. The tailoring step runs against the overridden document. The final package reflects the user's choice.
Severity:	Medium
Passed?:	Yes

Table 43: Test Case TF-43

5.1.44 Functional TF-44 : Auto apply cancellation at final review cleans up state

Test ID:	**TF-44**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that cancelling at the final review checkpoint does not leave a partial Application record.
Procedure:	1. Start the auto-apply workflow and proceed through both checkpoints. 2. At the final review screen, click Cancel. 3. Check the application tracker for any new records.
Outcome:	No Application record is created after cancellation. The application tracker shows no new entry for that opportunity. The workflow terminates cleanly and the user can restart the flow.
Severity:	Medium
Passed?:	Yes

Table 44: Test Case TF-44

5.1.45 Functional TF-45 : Auto apply submission path for internal opportunity

Test ID:	**TF-45**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that an auto-apply submission for an employer-posted opportunity is submitted through the internal flow.
Procedure:	1. Identify an opportunity created by a recruiter directly on UppGrad. 2. Start and complete the auto-apply workflow including final approval. 3. Check the application tracker and the recruiter applicant list.
Outcome:	The application is submitted through the internal UppGrad flow. An Application record is created with status submitted. The application appears on the recruiter's applicant list.
Severity:	High
Passed?:	Yes

Table 45: Test Case TF-45

5.1.46 Functional TF-46 : Auto apply fallback to manual package for external opportunity

Test ID:	**TF-46**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that for a scraped external opportunity with automation barriers the workflow packages materials for manual submission.
Procedure:	1. Identify a scraped opportunity with a CAPTCHA or SSO barrier. 2. Start and complete the auto-apply workflow. 3. Observe the submission stage output.
Outcome:	The workflow detects the automation barrier and routes to the manual-fallback path. The tailored package is presented with a clear message explaining why automatic submission was not attempted. No external POST is made.
Severity:	High
Passed?:	Yes

Table 46: Test Case TF-46

5.1.47 Functional TF-47 : Auto apply CAPTCHA detection stops automation

Test ID:	**TF-47**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the submission agent detects a CAPTCHA and refuses automated submission.
Procedure:	1. Configure or identify an external opportunity whose application page contains a CAPTCHA. 2. Run the auto-apply workflow through to the submission stage.
Outcome:	The submission agent classifies the target as non-automatable. No form submission is attempted. The user is presented with the prepared materials and instructed to submit manually.
Severity:	Medium
Passed?:	Yes

Table 47: Test Case TF-47

5.1.48 Functional TF-48 : Auto apply with missing required document in library

Test ID:	**TF-48**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that a missing required document type surfaces a clear pending state.
Procedure:	1. Ensure a cover letter is absent from the student document library. 2. Start auto-apply for an opportunity that requires a cover letter. 3. Observe the workflow output.
Outcome:	The workflow enters a pending state and surfaces a clear message listing the missing document type. The workflow does not continue to tailoring until the student provides the document.
Severity:	Medium
Passed?:	Yes

Table 48: Test Case TF-48

5.1.49 Functional TF-49 : Auto apply consent token required for each submission

Test ID:	**TF-49**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the submission stage requires a freshly issued consent token and cannot reuse a token from a previous run.
Procedure:	1. Complete one auto-apply run and note the consent token issued. 2. Attempt to call the submission endpoint with the same token a second time. 3. Observe the response.
Outcome:	The second submission attempt is rejected. The system reports that the consent token has already been consumed. A new workflow run with a new approval is required for any further submission.
Severity:	Medium
Passed?:	Yes

Table 49: Test Case TF-49

5.1.50 Functional TF-50 : Auto apply eligibility pending state with missing profile information

Test ID:	**TF-50**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that missing profile fields cause a pending interrupt that resumes after the student provides the information.
Procedure:	1. Use a student profile missing a required field such as work eligibility status. 2. Start the auto-apply workflow for an opportunity requiring that field. 3. Observe the pending interrupt. 4. Supply the missing information and resume the workflow.
Outcome:	The workflow pauses at a human checkpoint identifying the missing field. After the student provides the information and resumes, the workflow continues to the asset mapping phase without restarting from scratch.
Severity:	High
Passed?:	Yes

Table 50: Test Case TF-50

5.1.51 Functional TF-51 : Auto apply does not fabricate qualifications

Test ID:	**TF-51**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the tailoring step never adds skills, awards, or titles not present in the student's actual profile.
Procedure:	1. Start auto-apply for an opportunity listing advanced skills absent from the student profile. 2. Allow the workflow to complete the tailoring step. 3. Inspect the tailored CV and cover letter.
Outcome:	The tailored documents do not contain any skills, certifications, awards, or experiences absent from the student's actual profile. All added content is traceable to existing profile facts.
Severity:	Medium
Passed?:	Yes

Table 51: Test Case TF-51

5.1.52 Functional TF-52 : Auto apply two interrupt points are mandatory

Test ID:	**TF-52**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the auto-apply graph has no execution path that bypasses either mandatory human checkpoint.
Procedure:	1. Attempt to advance the graph past the asset mapping checkpoint without a user approval token. 2. Attempt to advance the graph past the final review checkpoint without a consent token.
Outcome:	Both advancement attempts are rejected. The graph does not transition to the next node without a valid approval token at each checkpoint. The submission agent never executes without the second checkpoint having been passed.
Severity:	Medium
Passed?:	Yes

Table 52: Test Case TF-52

5.1.53 Functional TF-53 : Auto apply notification sent when package is ready for review

Test ID:	**TF-53**
Category:	Functional
Sub-Category:	Auto Apply
Objective:	Verify that the student receives a notification when the auto-apply package reaches the final review checkpoint.
Procedure:	1. Start an auto-apply workflow run. 2. Allow the workflow to complete tailoring and reach the final review checkpoint. 3. Check in-app notifications and email.
Outcome:	A notification is dispatched when the workflow pauses at the final review checkpoint. The in-app notification is visible and identifies the opportunity. If email is enabled, an email is also sent.
Severity:	Medium
Passed?:	Yes

Table 53: Test Case TF-53

5.1.54 Functional TF-54 : Role based username or email label switching

Test ID:	**TF-54**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Username/Email label and placeholder text update based on the selected role.
Procedure:	1. Navigate to the UppGrad login page. 2. Click on the Student tab. 3. Click on the Employer tab.
Outcome:	Selecting Student shows the label Email and placeholder your.email@university.edu. Selecting Employer shows the label Username and placeholder your company username.
Severity:	Medium
Passed?:	Yes

Table 54: Test Case TF-54

5.1.55 Functional TF-55 : Password visibility toggle behaviour

Test ID:	**TF-55**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the functionality of the eye icon in the password field.
Procedure:	1. Enter a value into the Password field. 2. Observe the default masking. 3. Click the Eye icon on the right side of the password box.
Outcome:	Clicking the icon should toggle the password visibility between masked and plain text.
Severity:	Medium
Passed?:	Yes

Table 55: Test Case TF-55

5.1.56 Functional TF-56 : Remember me checkbox interaction

Test ID:	**TF-56**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the user can interact with the Remember me checkbox.
Procedure:	1. Click on the Remember me checkbox. 2. Refresh the page or navigate away and back.
Outcome:	The checkbox should be selectable and deselectable. If checked, the system should ideally retain the username upon the next visit.
Severity:	Medium
Passed?:	Yes

Table 56: Test Case TF-56

5.1.57 Functional TF-57 : Sign up and forgot password redirection

Test ID:	**TF-57**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Sign up and Forgot password links redirect to the correct pages.
Procedure:	1. Click the Forgot password link. 2. Go back and click the Sign up link at the bottom.
Outcome:	The user should be redirected to the password recovery page and the account registration page respectively.
Severity:	Medium
Passed?:	Yes

Table 57: Test Case TF-57

5.1.58 Functional TF-58 : Active role visual distinction on login

Test ID:	**TF-58**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the active role selection is visually distinct from inactive roles.
Procedure:	1. Observe the role selection section. 2. Click on each role one by one.
Outcome:	The active role should have a blue border and blue text, while inactive roles should have a light gray border and dark gray text.
Severity:	Low
Passed?:	Yes

Table 58: Test Case TF-58

5.1.59 Functional TF-59 : Empty field validation on sign in

Test ID:	**TF-59**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify system behaviour when clicking Sign In with empty fields.
Procedure:	1. Leave the Username/Email and Password fields empty. 2. Click the Sign In button.
Outcome:	Error messages should appear indicating that fields are required. The user should not be logged in.
Severity:	Medium
Passed?:	Yes

Table 59: Test Case TF-59

5.1.60 Functional TF-60 : Long and special character input handling in login

Test ID:	**TF-60**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the input fields handle maximum character lengths and special characters.
Procedure:	1. Select the Student role. 2. Enter a very long string (255+ characters) into the Email field. 3. Enter special characters or HTML tags into the Email field. 4. Click Sign In.
Outcome:	The UI should not break or overflow. The system should sanitize input to prevent Script Injection (XSS).
Severity:	Medium
Passed?:	Yes

Table 60: Test Case TF-60

5.1.61 Functional TF-61 : Password visibility consistency during role switching

Test ID:	**TF-61**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that toggling password visibility remains consistent when switching roles.
Procedure:	1. Select Student and enter a password. 2. Click the Eye icon to show the password in plain text. 3. Switch the role to Employer. 4. Observe the password field.
Outcome:	The password should revert to a masked state when the role is changed, or maintain the user's toggle preference consistently.
Severity:	High
Passed?:	Yes

Table 61: Test Case TF-61

5.1.62 Functional TF-62 : Credential retention during role switching

Test ID:	**TF-62**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify whether switching roles clears previously entered credentials.
Procedure:	1. Select Student and type an email and password. 2. Click on Recruiter. 3. Switch back to Student.
Outcome:	The system should follow the requirement document. Typically, switching roles should not lose user progress unless specified.
Severity:	Medium
Passed?:	Yes

Table 62: Test Case TF-62

5.1.63 Functional TF-63 : Enter key form submission on login

Test ID:	**TF-63**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the form is submitted when the user presses the Enter key.
Procedure:	1. Enter valid credentials in the Username and Password fields. 2. While the cursor is in the Password field, press the Enter key.
Outcome:	The system should trigger the Sign In action exactly as if the button was clicked.
Severity:	Medium
Passed?:	Yes

Table 63: Test Case TF-63

5.1.64 Functional TF-64 : Student sign up field and description display

Test ID:	**TF-64**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Student role displays the correct unique fields and descriptions.
Procedure:	1. Navigate to the Sign Up page. 2. Select the Student card. 3. Observe the card subtitle and available input fields.
Outcome:	Input fields for First Name, Last Name, Email, Password, and Confirm Password are visible. The placeholder for Email is your.email@university.edu.
Severity:	High
Passed?:	Yes

Table 64: Test Case TF-64

5.1.65 Functional TF-65 : Student legal consent and account creation action

Test ID:	**TF-65**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the final submission button and legal consent for Students.
Procedure:	1. Fill in all mandatory fields as a Student. 2. Click the I agree to the Terms of Service and Privacy Policy checkbox. 3. Observe the action button text.
Outcome:	The primary action button text is Create Account.
Severity:	High
Passed?:	Yes

Table 65: Test Case TF-65

5.1.66 Functional TF-66 : Employer sign up specialized fields and progress bar

Test ID:	**TF-66**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Employer role displays specialized corporate fields and a multi-step progress bar.
Procedure:	1. Navigate to the Sign Up page. 2. Select the Employer card. 3. Observe the form layout and progress indicator.
Outcome:	A progress bar shows step 1 Basic Info as Active and Verification as Inactive. Unique fields for Organization and Job Title are displayed. The subtitle Hiring talented students appears under the Employer label.
Severity:	Medium
Passed?:	Yes

Table 65: Test Case TF-65

5.1.67 Functional TF-67 : Employer multi-step registration flow

Test ID:	**TF-67**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the Employer role displays specialized corporate fields and a multi-step progress bar.
Procedure:	1. Navigate to the Sign Up page. 2. Select the Employer card. 3. Observe the form layout and progress indicator.
Outcome:	A progress bar shows step 1 Basic Info as Active and Verification as Inactive. Unique fields for Organization and Job Title are displayed. The subtitle Hiring talented students appears under the Employer label.
Severity:	Medium
Passed?:	Yes

Table 67: Test Case TF-67

5.1.68 Functional TF-68 : Password and confirm password mismatch validation

Test ID:	**TF-68**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the Password and Confirm Password synchronization.
Procedure:	1. Enter a valid password in the Password field. 2. Enter a different value in the Confirm Password field. 3. Attempt to proceed by clicking the primary action button.
Outcome:	The system must show a validation error indicating that passwords do not match.
Severity:	Medium
Passed?:	Yes

Table 68: Test Case TF-68

5.1.69 Functional TF-69 : Invalid credentials login attempt

Test ID:	**TF-69**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that incorrect credentials display an appropriate error without revealing whether the email or password is wrong.
Procedure:	1. Navigate to the login page. 2. Enter a valid email with an incorrect password. 3. Click Sign In. 4. Observe the error message.
Outcome:	The system displays a generic authentication error message. The user is not logged in. No session is created. The message does not specify whether the email or password is incorrect.
Severity:	Low
Passed?:	Yes

Table 69: Test Case TF-69

5.1.70 Functional TF-70 : Account lockout after repeated failed login attempts

Test ID:	**TF-70**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that the system enforces a lockout or rate-limit after a configurable number of consecutive failed login attempts.
Procedure:	1. Attempt to log in with incorrect credentials repeatedly (5 or more times). 2. Observe whether the system responds differently after the threshold is crossed.
Outcome:	After the threshold is exceeded the system either temporarily locks the account, adds a delay, or displays a too-many-attempts warning. Further login attempts are restricted until the lockout period expires or the user resets their password.
Severity:	High
Passed?:	Yes

Table 70: Test Case TF-70

5.1.71 Functional TF-71 : Email verification required before first login

Test ID:	**TF-71**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that a newly registered student cannot log in before confirming their email address.
Procedure:	1. Register a new student account with a valid email. 2. Attempt to log in immediately without clicking the verification link. 3. Observe the response.
Outcome:	The login is rejected with a message indicating that the email has not been verified. The verification email is sent. Login succeeds only after the link is clicked.
Severity:	High
Passed?:	Yes

Table 71: Test Case TF-71

5.1.72 Functional TF-72 : Forgot password flow sends email and allows reset

Test ID:	**TF-72**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify the complete forgot-password flow from link request to successful password change.
Procedure:	1. Click Forgot password and submit a registered email. 2. Open the reset link from the email. 3. Enter and confirm a new password. 4. Log in with the new password.
Outcome:	A password reset email is sent. The reset link opens a page that accepts a new password. After setting it, the old password no longer works and the new one allows login.
Severity:	Medium
Passed?:	Yes

Table 72: Test Case TF-72

5.1.73 Functional TF-73 : Session invalidated after sign out

Test ID:	**TF-73**
Category:	Functional
Sub-Category:	Login & Sign In
Objective:	Verify that signing out invalidates the session so subsequent requests with the old session cookie are rejected.
Procedure:	1. Log in as a student and copy the session cookie. 2. Click Sign Out. 3. Attempt to access a protected page using the old session cookie directly.
Outcome:	After sign out the session is invalidated server-side. Requests carrying the old cookie receive a redirect to the login page or a 401/403 response. The old session cannot be reused.
Severity:	Critical
Passed?:	Yes

Table 73: Test Case TF-73

5.1.74 Functional TF-74 : Unauthenticated direct profile access restriction

Test ID:	**TF-74**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that unauthenticated users cannot access the Profile page directly.
Procedure:	1. Log out of the system. 2. Enter the Profile page URL directly in the browser. 3. Attempt to load the page.
Outcome:	The user is redirected to the login page with no profile information being shown.
Severity:	Medium
Passed?:	Yes

Table 74: Test Case TF-74

5.1.75 Functional TF-75 : Profile retrieval for authenticated user

Test ID:	**TF-75**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that profile information is correctly retrieved from the database for a logged-in user.
Procedure:	1. Log in with a valid account. 2. Navigate to the Profile page. 3. Observe the displayed user information.
Outcome:	The system fetches and displays the correct profile information associated with the logged-in account.
Severity:	Medium
Passed?:	Yes

Table 75: Test Case TF-75

5.1.76 Functional TF-76 : Profile update persistence

Test ID:	**TF-76**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that profile data is correctly stored in the database after editing profile fields.
Procedure:	1. Log in and navigate to the Edit Profile functionality. 2. Modify a profile field. 3. Save the changes. 4. Refresh the profile page.
Outcome:	The updated profile information is correctly stored and retrieved from the database.
Severity:	Medium
Passed?:	Yes

Table 76: Test Case TF-76

5.1.77 Functional TF-77 : Extremely long profile input handling

Test ID:	**TF-77**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify system behaviour when profile fields contain extremely long input values.
Procedure:	1. Open the Edit Profile functionality. 2. Enter extremely long text in fields such as project description or skill entries. 3. Attempt to save the profile.
Outcome:	The system validates input length and prevents system failure, either rejecting the input with an appropriate error message or safely handling the input.
Severity:	Low
Passed?:	Yes

Table 77: Test Case TF-77

5.1.78 Functional TF-78 : Incomplete profile data handling

Test ID:	**TF-78**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that the system handles incomplete profile data without system errors.
Procedure:	1. Use an account with missing profile fields such as skills or experience. 2. Navigate to the Profile page.
Outcome:	The system loads the profile page successfully and handles missing data gracefully by showing appropriate messages about its absence.
Severity:	Medium
Passed?:	Yes

Table 78: Test Case TF-78

5.1.79 Functional TF-79 : Recruiter visibility control on profile sections

Test ID:	**TF-79**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that a student can control which profile sections are visible to recruiters.
Procedure:	1. Navigate to profile visibility settings. 2. Hide at least one profile section from recruiters. 3. Log in as a recruiter and view that student's profile via an application.
Outcome:	The section marked as hidden does not appear in the recruiter view. All other sections marked as visible are present. The student's visibility preferences are persisted.
Severity:	Medium
Passed?:	Yes

Table 79: Test Case TF-79

5.1.80 Functional TF-80 : Personal data export generation

Test ID:	**TF-80**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that a student can export personal platform data successfully.
Procedure:	1. Log in as a Student. 2. Navigate to account or privacy settings. 3. Trigger the personal data export action. 4. Wait for the export and open the result.
Outcome:	The system generates the student's data export successfully. The exported data contains only the requesting student's records.
Severity:	Low
Passed?:	Yes

Table 80: Test Case TF-80

5.1.81 Functional TF-81 : Profile CV import populates fields

Test ID:	**TF-81**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that importing a CV through the profile import flow parses and populates the relevant profile fields correctly.
Procedure:	1. Navigate to the profile import or CV upload section. 2. Upload a valid CV. 3. Observe the resulting profile fields.
Outcome:	Education, experience, skills, and other structured fields present in the CV are populated in the corresponding profile sections. The user is shown the parsed results for review before saving.
Severity:	Medium
Passed?:	Yes

Table 81: Test Case TF-81

5.1.82 Functional TF-82 : First and last name persisted at registration

Test ID:	**TF-82**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that deleting an account triggers removal or anonymisation of personal data within the required retention window.
Procedure:	1. Log in as a student and trigger the account deletion flow. 2. Confirm the deletion. 3. After 7 days or by simulating the scheduled task, inspect the database for remaining personal data.
Outcome:	Personal data such as name, email, and profile content is removed or anonymised within 7 days. Application records that must be retained legally are preserved but unlinked from identifiable information. The deleted account cannot be logged into.
Severity:	Medium
Passed?:	Yes

Table 82: Test Case TF-82

5.1.83 Functional TF-83 : Account deletion anonymises personal data

Test ID:	**TF-83**
Category:	Functional
Sub-Category:	Profile
Objective:	Verify that deleting an account triggers removal or anonymisation of personal data within the required retention window.
Procedure:	1. Log in as a student and trigger the account deletion flow. 2. Confirm the deletion. 3. After 7 days or by simulating the scheduled task, inspect the database for remaining personal data.
Outcome:	Personal data such as name, email, and profile content is removed or anonymised within 7 days. Application records that must be retained legally are preserved but unlinked from identifiable information. The deleted account cannot be logged into.
Severity:	High
Passed?:	Yes

Table 83: Test Case TF-83

5.1.84 Functional TF-84 : Recruiter can move applicant through pipeline stages

Test ID:	**TF-84**
Category:	Functional
Sub-Category:	Recruiter Management
Objective:	Verify that a recruiter can move an applicant through the hiring pipeline stages and that each transition is recorded correctly.
Procedure:	1. Log in as a Recruiter and navigate to a posting with at least one applicant. 2. Move the applicant from Submitted to Under Review. 3. Move the applicant again from Under Review to Interview Scheduled. 4. Check the applicant's status in the system.
Outcome:	Each stage transition is recorded correctly and reflected immediately in the recruiter's applicant view. The applicant's current stage is accurate at each step. The status history is preserved and does not overwrite previous transitions.
Severity:	Medium
Passed?:	Yes

Table 84: Test Case TF-84

5.1.85 Functional TF-85 : Recruiter can add private notes to a candidate

Test ID:	**TF-85**
Category:	Functional
Sub-Category:	Recruiter Management
Objective:	Verify that a recruiter can add private notes to a candidate's application and that these notes are not visible to the student.
Procedure:	1. Log in as a Recruiter and open an applicant's profile for a specific posting. 2. Add a private note to the applicant. 3. Save the note and refresh the page. 4. Log in as the student and check whether the note is visible anywhere in their view.
Outcome:	The private note is saved and visible only to the recruiter. The note persists after a page refresh. The student has no access to the note content through the student-facing interface or API.
Severity:	High
Passed?:	Yes

Table 85: Test Case TF-85

5.1.86 Functional TF-86 : Recruiter restricted to own organisation data

Test ID:	**TF-86**
Category:	Functional
Sub-Category:	Recruiter Management
Objective:	Verify that a recruiter cannot access postings, applicants, or analytics belonging to a different organisation.
Procedure:	1. Log in as Recruiter A from Organisation A. 2. Attempt to access the postings and applicant list of Organisation B by manipulating URL parameters or API calls. 3. Observe the system response.
Outcome:	All cross-organisation access attempts are rejected with a 403 or 404 response. Recruiter A sees only data belonging to Organisation A. No data from Organisation B is returned in any API response or rendered in the interface.
Severity:	Medium
Passed?:	Yes

Table 86: Test Case TF-86

5.1.87 Functional TF-87 : Student can manually track an external application

Test ID:	**TF-87**
Category:	Functional
Sub-Category:	Application Tracking
Objective:	Verify that a student can manually add and track an application to an opportunity that was not submitted through UppGrad.
Procedure:	1. Log in as a student and navigate to the application tracker. 2. Add a manual application entry for an external opportunity with a custom status. 3. Save the entry and navigate away. 4. Return to the application tracker and locate the entry.
Outcome:	The manually tracked application appears in the application tracker with the correct status. The entry persists after navigation. The student can update the status and add notes to the entry independently of any UppGrad submission flow.
Severity:	Medium
Passed?:	Yes

Table 87: Test Case TF-87

5.1.88 Functional TF-88 : Application status updated manually by student

Test ID:	**TF-88**
Category:	Functional
Sub-Category:	Application Tracking
Objective:	Verify that a student can manually update the status of a tracked application through the available stages.
Procedure:	1. Log in as a student and navigate to the application tracker. 2. Open an existing application entry. 3. Change the status from Submitted to Interview. 4. Save the change and refresh the page.
Outcome:	The updated status is saved and displayed correctly after refresh. The status change is reflected in the application history. The student can cycle through all available stages without errors.
Severity:	High
Passed?:	Yes

Table 88: Test Case TF-88

5.1.89 Functional TF-89 : Student can add private notes and reminders to an application

Test ID:	**TF-89**
Category:	Functional
Sub-Category:	Application Tracking
Objective:	Verify that a student can attach private notes and reminders to an application entry and that these persist correctly.
Procedure:	1. Log in as a student and open an application entry in the tracker. 2. Add a private note with free text. 3. Add a reminder with a future date. 4. Save and navigate away, then return to the entry.
Outcome:	The private note and reminder are both saved and visible when the student returns to the entry. The reminder date is stored correctly. Neither the note nor the reminder is visible to the recruiter or any other user.
Severity:	Low
Passed?:	Yes

Table 89: Test Case TF-89

5.1.90 Functional TF-90 : Opportunity filter by type returns correct results

Test ID:	**TF-90**
Category:	Functional
Sub-Category:	Opportunity Discovery
Objective:	Verify that filtering opportunities by type returns only results matching the selected type.
Procedure:	1. Log in as a student and navigate to the opportunity feed. 2. Apply a filter for a specific opportunity type such as Internship. 3. Observe the returned results. 4. Apply a different filter for Scholarship and observe the results.
Outcome:	Only opportunities matching the selected type are shown in the feed. Switching the filter immediately updates the results. No opportunities of a different type appear in the filtered view.
Severity:	Medium
Passed?:	Yes

Table 90: Test Case TF-90

5.1.91 Functional TF-91 : Opportunity filter by location returns correct results

Test ID:	**TF-91**
Category:	Functional
Sub-Category:	Opportunity Discovery
Objective:	Verify that the system sends a deadline reminder notification to a student for a bookmarked opportunity whose deadline is within 24 hours.
Procedure:	1. Log in as a student and ensure deadline reminders are enabled in notification settings. 2. Bookmark an opportunity with a deadline set to within 24 hours from the current time. 3. Trigger the send deadline reminders background task. 4. Check the student's in-app notifications and email.
Outcome:	A notification is generated and delivered alerting the student about the impending deadline. The notification identifies the opportunity by name. If email notifications are enabled an email is also sent. The notification is not duplicated if the task runs more than once.
Severity:	Low
Passed?:	Yes

Table 91: Test Case TF-91

5.1.92 Functional TF-92 : Deadline reminder notification delivered for bookmarked opportunity

Test ID:	**TF-92**
Category:	Functional
Sub-Category:	Notifications
Objective:	Verify that the system sends a deadline reminder notification to a student for a bookmarked opportunity whose deadline is within 24 hours.
Procedure:	1. Log in as a student and ensure reminders are enabled in notification settings. 2. Bookmark an opportunity with a deadline set to within 24 hours from the time. 3. Trigger the send deadline reminders background task. 4. Check the student's in-app notifications and email.
Outcome:	A notification is generated and delivered alerting the student about the impending deadline. The notification identifies the opportunity by name. If email notifications are enabled an email is also sent. The notification is not duplicated if the task runs more than once.
Severity:	High
Passed?:	Yes

Table 92: Test Case TF-92

5.1.93 Functional TF-93 : Notification preferences respected per category

Test ID:	**TF-93**
Category:	Functional
Sub-Category:	Notifications
Objective:	Verify that disabling a specific notification category in the student's notification settings prevents notifications of that category from being delivered.
Procedure:	1. Log in as a student and navigate to notification settings. 2. Disable the deadline reminders notification category. 3. Trigger an event that would normally generate a deadline reminder. 4. Check in-app notifications and email for any delivered reminder.
Outcome:	No deadline reminder notification is delivered after the category is disabled. Other notification categories that are still enabled continue to function correctly. Re-enabling the category restores delivery of that notification type.
Severity:	Medium
Passed?:	Yes

Table 93: Test Case TF-93

5.1.94 Functional TF-94 : Hidden scraped opportunity before approval

Test ID:	**TF-94**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that scraped opportunities remain hidden until reviewed or activated by authorised staff.
Procedure:	1. Log in as University Staff. 2. Scrape an opportunity from an external source. 3. Leave the scraped opportunity unapproved or inactive. 4. Log in as a Student and look for it. 5. Log in as University Staff and activate the opportunity. 6. Log in as a Student and look for the opportunity again.
Outcome:	The student should not be able to see the opportunity until the staff approves and activates it. Afterwards, the student should see the opportunity.
Severity:	Medium
Passed?:	Yes

Table 94: Test Case TF-94

5.1.95 Functional TF-95 : Closed scraped opportunity not shown in feed

Test ID:	**TF-95**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that an opportunity marked as closed by the Closed Checker sub-service is removed from the student-facing feed.
Procedure:	1. Identify a previously active scraped opportunity. 2. Run or simulate the Closed Checker to mark it as closed. 3. Log in as a student and search for the opportunity.
Outcome:	The closed opportunity does not appear in the student opportunity feed or search results. Any existing bookmark for that opportunity is flagged as closed in the student's bookmark list.
Severity:	High
Passed?:	Yes

Table 95: Test Case TF-95

5.1.96 Functional TF-96 : Scraped opportunity normalised into unified schema

Test ID:	**TF-96**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that a scraped opportunity conforms to the unified opportunity schema and appears alongside recruiter-created opportunities in the feed.
Procedure:	1. Trigger or simulate a scraper run ingesting a new job listing. 2. Log in as a student and open the opportunity feed. 3. Locate the scraped opportunity.
Outcome:	The scraped opportunity appears in the unified feed alongside recruiter-created postings. All required schema fields are populated. The opportunity can be bookmarked and applied to through an external link.
Severity:	Medium
Passed?:	Yes

Table 96: Test Case TF-96

5.1.97 Functional TF-97 : Duplicate scraped opportunity not created

Test ID:	**TF-97**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that running the scraper twice does not create duplicate entries for the same external posting.
Procedure:	1. Trigger the scraper once and confirm a specific opportunity is ingested. 2. Trigger the scraper again for the same source and date range. 3. Query the database for the opportunity.
Outcome:	Only one row exists for the opportunity after two scraper runs. The second run detects the existing record and does not insert a duplicate.
Severity:	High
Passed?:	Yes

Table 97: Test Case TF-97

5.1.98 Functional TF-98 : Scraper output readable by recommendation engine without manual sync

Test ID:	**TF-98**
Category:	Functional
Sub-Category:	Scraping
Objective:	Verify that a newly ingested scraped opportunity is picked up by the recommendation engine on the next refresh without any manual synchronisation step.
Procedure:	1. Ingest a new opportunity via the scraper. 2. Trigger a recommendation refresh for a student whose profile matches the opportunity. 3. Check the student's recommendation feed.
Outcome:	The newly ingested opportunity appears in the student's recommendation feed after the refresh. No manual data synchronisation step was required between the scraper and the recommendation engine.
Severity:	Medium
Passed?:	Yes

Table 98: Test Case TF-98

5.1.99 Functional TF-99 : Recommendation refresh after job edit

Test ID:	**TF-99**
Category:	Functional
Sub-Category:	AI Workflow
Objective:	Verify that editing a job posting triggers recommendation recalculation and student notification.
Procedure:	1. Log in as a Recruiter and open an existing job posting. 2. Edit fields such as requirements, skills, or deadline. 3. Save the posting. 4. Log in as an affected Student and check recommendations and notifications.
Outcome:	The system saves the updated posting, recalculates affected recommendation scores, reorders recommendations where needed, and generates a notification for impacted students.
Severity:	Critical
Passed?:	Yes

Table 99: Test Case TF-99

5.1.100 Functional TF-100 : Long running AI workflow survives backend restart

Test ID:	**TF-100**
Category:	Functional
Sub-Category:	AI Workflow
Objective:	Verify that a document feedback or auto-apply workflow mid-execution can be resumed after a backend worker restart.
Procedure:	1. Start a document feedback workflow and allow it to reach the human review checkpoint. 2. Restart the backend worker process. 3. Return to the review page and attempt to resume the workflow.
Outcome:	The workflow state is persisted to durable storage before the worker restart. After the restart, the review page loads the saved state and the user can continue from the checkpoint without losing progress.
Severity:	High
Passed?:	Yes

Table 100: Test Case TF-100

5.1.101 Functional TF-101 : Transient LLM failure does not corrupt session

Test ID:	**TF-101**
Category:	Functional
Sub-Category:	AI Workflow
Objective:	Verify that a transient LLM provider error does not leave an ApplicationSession record in a corrupt or inconsistent state.
Procedure:	1. Simulate a transient LLM provider error during the synthesis phase of document feedback. 2. Observe the session status. 3. Retry the workflow.
Outcome:	The session status transitions to a failed or retryable state rather than hanging indefinitely. No partial or corrupt feedback record is persisted. The user sees a clear error message. Retrying succeeds when the provider recovers.
Severity:	Medium
Passed?:	Yes

Table 101: Test Case TF-101

5.1.102 Functional TF-102 : Application status change triggers student notification

Test ID:	**TF-102**
Category:	Functional
Sub-Category:	AI Workflow
Objective:	Verify that a recruiter moving an applicant to a new pipeline stage triggers a notification to the student.
Procedure:	1. Log in as a Recruiter and open a posting with at least one applicant. 2. Move the applicant from Submitted to Under Review. 3. Log in as the student and check notifications.
Outcome:	A notification is created for the student reflecting the status change. If email notifications are enabled, an email is dispatched. The in-app notification is visible and identifies the posting.
Severity:	Medium
Passed?:	Yes

Table 102: Test Case TF-102

5.1.103 Functional TF-103 : Weights sum invariant in matching algorithm

Test ID:	**TF-103**
Category:	Functional
Sub-Category:	AI Workflow
Objective:	Verify that the matching algorithm scoring weights are normalised and sum to one.
Procedure:	1. Retrieve the current scoring weight configuration. 2. Sum all configured weights. 3. Modify one weight and verify the system re-normalises or rejects the invalid configuration.
Outcome:	The sum of all scoring weights is exactly one. If a configuration update would break this invariant, the system rejects it with a clear validation error. Recommendations are never computed with un-normalised weights.
Severity:	Medium
Passed?:	Yes

Table 103: Test Case TF-103

5.1.104 Functional TF-104 : Profile-based recommendation generation

Test ID:	**TF-104**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify AI recommendations are generated and displayed based on the student's profile.
Procedure:	1. Log in as a Student with a fully populated profile including skills, education, and preferences. 2. Navigate to the Student Home Page or Dashboard. 3. Observe the generated recommendation cards.
Outcome:	Recommendations should be successfully fetched and should match the student's specific skills and preferences, for example showing Software Engineering jobs for a CS student.
Severity:	Medium
Passed?:	Yes

Table 104: Test Case TF-104

5.1.105 Functional TF-105 : Unified mixed opportunity recommendation feed

Test ID:	**TF-105**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that the unified recommendation feed includes mixed opportunity types.
Procedure:	1. Log in as a Student. 2. Navigate to the Student Home Page. 3. Scroll through the AI recommendation feed.
Outcome:	The feed must seamlessly display a mix of Job Postings, Master's Programs, Scholarships, and other opportunities, indicating that the unified graph logic works.
Severity:	High
Passed?:	Yes

Table 105: Test Case TF-105

5.1.106 Functional TF-106 : Recommendation refresh after profile change

Test ID:	**TF-106**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that changes to a student's profile trigger a complete refresh of their AI-generated recommendation feed.
Procedure:	1. Log in as a Student and note the current mix of opportunities in the AI recommendation feed. 2. Navigate to Edit Profile and make significant changes such as replacing Software Engineering with Data Science. 3. Save the profile changes and return to the Home Page recommendation feed.
Outcome:	The system detects the profile update, triggers the asynchronous recommendation refresh, and the feed repopulates with new opportunities relevant to the updated profile.
Severity:	High
Passed?:	Yes

Table 106: Test Case TF-106

5.1.107 Functional TF-107 : Recommendation explanation provided for each item

Test ID:	**TF-107**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that each recommended opportunity includes a human-readable explanation of why it was suggested.
Procedure:	1. Log in as a student with a populated profile. 2. Navigate to the recommendation feed. 3. Open the detail view for at least three recommended opportunities. 4. Locate the explanation text for each.
Outcome:	Each recommendation includes a short plain-language explanation identifying the primary matching factors such as matching skills, location fit, or degree level. No recommendation is shown without any explanation.
Severity:	Medium
Passed?:	Yes

Table 107: Test Case TF-107

5.1.108 Functional TF-108 : Recommendation feed refreshed for inactive user after 7 days

Test ID:	**TF-108**
Category:	Functional
Sub-Category:	AI Recommendations
Objective:	Verify that the recommendation feed is automatically refreshed for a student inactive for 7 or more days.
Procedure:	1. Create a student account and generate an initial recommendation feed. 2. Simulate or wait for 7 days of inactivity. 3. Trigger the scheduled Celery Beat task for stale feed refresh. 4. Log in and observe the feed.
Outcome:	The feed is refreshed and newly ingested matching opportunities appear. The feed timestamp reflects the recent refresh. The process completes without the student needing to log in to trigger it.
Severity:	Medium
Passed?:	Yes

Table 108: Test Case TF-108

5.1.109 Functional TF-109 : Smart Rank applicant reordering

Test ID:	**TF-109**
Category:	Functional
Sub-Category:	Smart Ranking
Objective:	Verify the Smart Rank functionality successfully reorders job applicants.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Open the Applicants list for a job posting with multiple applicants. 3. Click the Smart Rank toggle.
Outcome:	The system evaluates applicant resumes against the job description using the AI endpoint and dynamically resorts the applicants from highest match score to lowest.
Severity:	High
Passed?:	Yes

Table 109: Test Case TF-109

5.1.110 Functional TF-110 : Smart Rank scores not based on protected characteristics

Test ID:	**TF-110**
Category:	Functional
Sub-Category:	Smart Ranking
Objective:	Verify that the Smart Rank scoring does not use protected characteristics such as age, gender, or ethnicity as input features.
Procedure:	1. Inspect the input features passed to the matching algorithm for Smart Rank. 2. Confirm that fields corresponding to protected characteristics are absent from the scoring inputs. 3. Run Smart Rank with two candidate profiles identical in skills but differing in demographic information.
Outcome:	The two demographically different but professionally identical profiles receive the same or statistically indistinguishable scores. Protected characteristic fields are not present in the scoring input.
Severity:	Medium
Passed?:	Yes

Table 110: Test Case TF-110

5.1.111 Functional TF-111 : Smart Rank score consistent with opportunity matching

Test ID:	**TF-111**
Category:	Functional
Sub-Category:	Smart Ranking
Objective:	Verify that Smart Rank scores are computed by the same matching algorithm used for student-side recommendations applied in reverse.
Procedure:	1. Identify a student whose profile has been scored by the recommendation engine for a given posting. 2. Enable Smart Rank for the same posting. 3. Compare the Smart Rank score shown for the applicant with the recommendation engine score.
Outcome:	The Smart Rank score is consistent with the recommendation engine's output when applied in the reverse direction. No separate or incompatible scoring logic is used for the recruiter view.
Severity:	High
Passed?:	Yes

Table 111: Test Case TF-111

5.1.112 Functional TF-112 : Employer job creation and search visibility

Test ID:	**TF-112**
Category:	Functional
Sub-Category:	Opportunity Management
Objective:	Verify employers can create jobs that appear in the global search.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Create a new Job Posting, filling in all required fields, and set it to active. 3. Log in as a Student and search for the exact job title.
Outcome:	The job posting is instantly persisted in the database and is discoverable in the student's search results.
Severity:	Medium
Passed?:	Yes

Table 112: Test Case TF-112

5.1.113 Functional TF-113 : Posting analytics conversion calculation

Test ID:	**TF-113**
Category:	Functional
Sub-Category:	Opportunity Management
Objective:	Verify the Job Analytics popup accurately calculates application conversion rates.
Procedure:	1. Log in as an Employer and navigate to the Employer Postings Page. 2. Open the Posting Analytics Popup for a job with a known number of views and applicants.
Outcome:	The system correctly aggregates total views and unique applicants, calculating the exact conversion percentage accurately.
Severity:	High
Passed?:	Yes

Table 113: Test Case TF-113

5.1.114 Functional TF-114 : Bookmark added to list successfully

Test ID:	**TF-114**
Category:	Functional
Sub-Category:	Bookmarks
Objective:	Verify that a student can bookmark an opportunity and have it appear in their bookmark list.
Procedure:	1. Log in as a student. 2. Navigate to the opportunity feed and open an opportunity. 3. Click the bookmark icon on the opportunity. 4. Navigate to the bookmark list.
Outcome:	The bookmarked opportunity appears in the student's bookmark list. The bookmark icon on the opportunity reflects the saved state. The bookmark persists after the page is refreshed.
Severity:	Low
Passed?:	Yes

Table 114: Test Case TF-114

5.1.115 Functional TF-115 : Bookmark removed when toggled off

Test ID:	**TF-115**
Category:	Functional
Sub-Category:	Bookmarks
Objective:	Verify that a student can remove a bookmark by toggling it off and that the opportunity is removed from the bookmark list.
Procedure:	1. Log in as a student with at least one bookmarked opportunity. 2. Navigate to the bookmark list. 3. Click the bookmark icon on a saved opportunity to remove it. 4. Observe the bookmark list after removal.
Outcome:	The opportunity is removed from the bookmark list immediately after the toggle. The bookmark icon on the opportunity returns to its unsaved state. The removal persists after the page is refreshed.
Severity:	Low
Passed?:	Yes

Table 115: Test Case TF-115

5.1.116 Functional TF-116 : Bookmarked opportunity flagged as closed

Test ID:	**TF-116**
Category:	Functional
Sub-Category:	Bookmarks
Objective:	Verify that when a bookmarked opportunity is closed or expired it is clearly flagged as such in the student's bookmark list rather than silently disappearing or appearing still active.
Procedure:	1. Log in as a student and bookmark an active opportunity. 2. Close or expire the opportunity from the recruiter or staff side. 3. Log back in as the student and navigate to the bookmark list.
Outcome:	The opportunity remains visible in the bookmark list but is clearly marked as closed or expired. The student is not misled into believing the opportunity is still active. The flag is applied without requiring the student to refresh manually.
Severity:	Medium
Passed?:	Yes

Table 116: Test Case TF-116

5.1.117 Functional TF-117 : Bookmark list organised into custom lists

Test ID:	**TF-117**
Category:	Functional
Sub-Category:	Bookmarks
Objective:	Verify that a student can organise bookmarked opportunities into named custom lists and that the organisation persists correctly.
Procedure:	1. Log in as a student and bookmark at least three opportunities. 2. Create two named custom lists such as Internships and Masters Programs. 3. Assign each bookmarked opportunity to one of the custom lists. 4. Navigate away and return to the bookmark list.
Outcome:	Each opportunity appears under the correct custom list. The list names and assignments persist after navigation. Opportunities not assigned to a custom list remain accessible in a default or uncategorised view.
Severity:	Low
Passed?:	Yes

Table 117: Test Case TF-117

5.2 Non-Functional Test Cases

5.2.1 Non-Functional TN-01 : Login page responsiveness on smaller screens

Test ID:	**TN-1**
Category:	Non-Functional
Sub-Category:	Responsiveness
Objective:	Verify the login card scales correctly on smaller resolutions.
Procedure:	1. Open the login page. 2. Reduce the browser window width to mobile dimensions. 3. Observe the role buttons.
Outcome:	The login card should resize without losing content. The role buttons should ideally stack or remain within the container borders.
Severity:	Low
Passed?:	Yes

Table 118: Test Case TN-1

5.2.2 Non-Functional TN-02 : Dashboard responsive layout on tablet viewport

Test ID:	**TN-2**
Category:	Non-Functional
Sub-Category:	Responsiveness
Objective:	Verify that the student dashboard renders correctly on a tablet-width viewport of 768px.
Procedure:	1. Log in as a student. 2. Resize the browser to 768px width. 3. Scroll through the dashboard including the recommendation feed and navigation.
Outcome:	All dashboard elements are visible and usable. No content overflows its container. Navigation items collapse or reorganise appropriately. The recommendation feed cards remain readable.
Severity:	Low
Passed?:	Yes

Table 119: Test Case TN-2

5.2.3 Non-Functional TN-03 : Password field copy restriction

Test ID:	**TN-3**
Category:	Non-Functional
Sub-Category:	Security
Objective:	Verify that the Copy function is disabled for the password field.
Procedure:	1. Enter a password. 2. Highlight the masked password and attempt to Copy using Ctrl+C. 3. Attempt to paste it into a notepad.
Outcome:	The user should not be able to copy the password from the masked field.
Severity:	Low
Passed?:	Yes

Table 120: Test Case TN-3

5.2.4 Non-Functional TN-04 : CSRF protection on state-changing endpoints

Test ID:	**TN-4**
Category:	Non-Functional
Sub-Category:	Security
Objective:	Verify that all state-changing API endpoints enforce CSRF protection.
Procedure:	1. Send a POST request to a protected endpoint without a valid CSRF token. 2. Repeat with an invalid token. 3. Repeat with a valid token for a different session.
Outcome:	All three requests are rejected with a 403 response. Only requests carrying a valid CSRF token for the current session are accepted.
Severity:	High
Passed?:	Yes

Table 121: Test Case TN-4

5.2.5 Non-Functional TN-05 : Documents served only to authorised users via signed URLs

Test ID:	**TN-5**
Category:	Non-Functional
Sub-Category:	Security
Objective:	Verify that uploaded documents are not accessible via direct public URLs and can only be retrieved through signed short-lived URLs issued to the owner.
Procedure:	1. Upload a document as Student A. 2. Obtain the signed URL issued for that document. 3. Attempt to access the signed URL after it has expired. 4. Attempt to access the file path directly without a signed URL.
Outcome:	The expired signed URL returns an access-denied response. Direct path access without a signed URL is blocked. Only a fresh signed URL issued to the owner grants access.
Severity:	Medium
Passed?:	Yes

Table 122: Test Case TN-5

5.2.6 Non-Functional TN-06 : Password stored with memory-hard hashing

Test ID:	**TN-6**
Category:	Non-Functional
Sub-Category:	Security
Objective:	Verify that user passwords are stored using a memory-hard hashing algorithm rather than plain text or weak hashing.
Procedure:	1. Register a new user account. 2. Inspect the stored password hash in the database.
Outcome:	The password is not stored in plain text. The stored value is a hash produced by a memory-hard algorithm such as Argon2. The hash is not reversible to the original password.
Severity:	High
Passed?:	Yes

Table 123: Test Case TN-6

5.2.7 Non-Functional TN-07 : Sign in redirection consistency and accessibility

Test ID:	**TN-7**
Category:	Non-Functional
Sub-Category:	Accessibility
Objective:	Verify visual consistency and accessibility of the Sign in redirection.
Procedure:	1. Scroll to the bottom of the Student or Employer Sign Up page. 2. Locate the Already have an account text. 3. Click the Sign in link.
Outcome:	The user is immediately redirected to the main Login page.
Severity:	Medium
Passed?:	Yes

Table 124: Test Case TN-7

5.2.8 Non-Functional TN-08 : Login page keyboard navigation accessibility

Test ID:	**TN-8**
Category:	Non-Functional
Sub-Category:	Accessibility
Objective:	Verify that the login page is fully navigable by keyboard alone.
Procedure:	1. Load the login page. 2. Navigate all interactive elements using only Tab, Shift+Tab, Enter, and Space. 3. Attempt to log in using only keyboard input.
Outcome:	Every interactive element is reachable and operable via keyboard. Focus indicators are clearly visible at each step. The login action can be completed without a mouse.
Severity:	Medium
Passed?:	Yes

Table 125: Test Case TN-8

5.2.9 Non-Functional TN-09 : Posting analytics consistency after stage updates

Test ID:	TN-9
Category:	Non-Functional
Sub-Category:	Reliability
Objective:	Verify that posting analytics update correctly after candidate stage changes.
Procedure:	1. Log in as a Recruiter. 2. Open a posting with multiple applicants and view its analytics section. 3. Move one or more candidates between pipeline stages. 4. Refresh the analytics view.
Outcome:	The analytics reflect the updated number of applicants in each stage. Stage conversion values are updated accordingly. The totals remain consistent with the current pipeline data.
Severity:	Medium
Passed?:	Yes

Table 126: Test Case TN-9

5.2.10 Non-Functional TN-10 : Nightly database backup and weekly restorability check

Test ID:	TN-10
Category:	Non-Functional
Sub-Category:	Reliability
Objective:	Verify that the automated nightly database backup runs successfully and that the backup can be restored into a disposable environment.
Procedure:	1. Confirm the nightly backup task ran without error. 2. Restore the most recent backup into an isolated test environment. 3. Verify that core tables and record counts match the production snapshot.
Outcome:	The backup completes without errors. The restore into the test environment succeeds. Core table row counts and spot-checked records match the source snapshot.
Severity:	High
Passed?:	Yes

Table 127: Test Case TN-10

5.2.11 Non-Functional TN-11 : Worker pool scales independently per job type

Test ID:	TN-11
Category:	Non-Functional
Sub-Category:	Reliability
Objective:	Verify that the worker count for each background job type can be tuned independently without redeploying the application.
Procedure:	1. Identify the worker configuration for recommendation, document feedback, auto apply, scraping, and email dispatch. 2. Change the worker count for one job type. 3. Verify the change takes effect without redeploying the application code.
Outcome:	The worker count change takes effect at the next worker start cycle without a code redeploy. Other worker pools are unaffected. The updated configuration is reflected in the health or metrics endpoint.
Severity:	Medium
Passed?:	Yes

Table 128: Test Case TN-11

5.2.12 Non-Functional TN-12 : Document page performance with many uploaded files

Test ID:	TN-12
Category:	Non-Functional
Sub-Category:	Performance
Objective:	Verify system performance when multiple documents exist for a user.
Procedure:	1. Upload or simulate a large number of documents for one user (e.g. 50+). 2. Load the Document Feedback page.
Outcome:	The document list loads successfully without excessive delays or system failures.
Severity:	Medium
Passed?:	Yes

Table 129: Test Case TN-12

5.2.13 Non-Functional TN-13 : Student dashboard initial render under 3 seconds

Test ID:	**TN-13**
Category:	Non-Functional
Sub-Category:	Performance
Objective:	Verify that the student dashboard initially renders within 3 seconds under normal conditions.
Procedure:	1. Log in as a student with a fully populated profile and an existing recommendation feed. 2. Navigate to the dashboard and measure time to first meaningful paint. 3. Repeat the measurement three times.
Outcome:	All three measurements show first meaningful paint within 3 seconds. Long-running computations such as recommendation recalculation do not block the initial render.
Severity:	High
Passed?:	Yes

Table 130: Test Case TN-13

5.2.14 Non-Functional TN-14 : Opportunity search returns results within 1.5 seconds

Test ID:	**TN-14**
Category:	Non-Functional
Sub-Category:	Performance
Objective:	Verify that the opportunity search endpoint responds within 1.5 seconds for moderate result sizes on a warm database.
Procedure:	1. Submit a search query that returns between 20 and 100 results. 2. Measure the API response time from request to first byte. 3. Repeat with three different queries.
Outcome:	All measured response times are below 1.5 seconds. Results are returned ranked and ready to display.
Severity:	High
Passed?:	Yes

Table 131: Test Case TN-14

5.2.15 Non-Functional TN-15 : Authentication endpoints respond within 2 seconds

Test ID:	**TN-15**
Category:	Non-Functional
Sub-Category:	Performance
Objective:	Verify that authentication endpoints respond within 2 seconds under normal load.
Procedure:	. Submit a login request with valid credentials. 2. Measure the response time. 3. Repeat for registration and password reset endpoints.
Outcome:	All authentication endpoint responses are received within 2 seconds. Memory-hard password hashing does not push responses beyond the threshold.
Severity:	High
Passed?:	Yes

Table 132: Test Case TN-15

5.2.16 Non-Functional TN-16 : Document status visibility and usability

Test ID:	**TN-16**
Category:	Non-Functional
Sub-Category:	Usability
Objective:	Verify that users can easily identify the status of their uploaded documents.
Procedure:	1. Navigate to Document Feedback. 2. Upload a document. 3. Observe the document entry in the document list.
Outcome:	The document clearly displays a status indicator such as Loading Document, Pending Feedback, or Reviewed.
Severity:	Medium
Passed?:	Yes

Table 133: Test Case TN-16

5.2.17 Non-Functional TN-17 : New student completes onboarding within 5 minutes

Test ID:	**TN-17**
Category:	Non-Functional
Sub-Category:	Usability
Objective:	Verify that a new student user can complete registration, profile setup, and first opportunity interaction within 5 minutes during moderated testing.
Procedure:	1. Recruit a new test user with no prior exposure to the platform. 2. Start a timer and have the user register, complete the profile setup wizard, and bookmark one opportunity. 3. Record the elapsed time.
Outcome:	The complete sequence is finished within 5 minutes. The user does not get stuck or require help at any step. The onboarding path contains no dead ends or confusing redirections.
Severity:	Medium
Passed?:	Yes

Table 134: Test Case TN-17

5.2.18 Non-Functional TN-18 : AI output presented as suggestions not final decisions

Test ID:	**TN-18**
Category:	Non-Functional
Sub-Category:	Usability
Objective:	Verify that AI-generated content is presented as suggestions for review rather than as final, unquestionable output.
Procedure:	1. Run the document feedback workflow. 2. Inspect the review screen for labelling and controls. 3. Run the auto-apply workflow and inspect both checkpoint screens.
Outcome:	The review screens label AI output clearly as proposed changes or suggestions. Users can reject individual proposals. No AI output is applied without explicit user acceptance.
Severity:	Medium
Passed?:	Yes

Table 135: Test Case TN-18

5.2.19 Non-Functional TN-19 : Cross-browser document upload compatibility

Test ID:	**TN-19**
Category:	Non-Functional
Sub-Category:	Compatibility
Objective:	Verify that document uploads work correctly across different major browsers.
Procedure:	1. Open the application in Chrome, Firefox, and Safari. 2. Navigate to the Document Feedback page. 3. Upload a document from each browser.
Outcome:	All core flows complete successfully on the mobile viewport. No controls are obscured or inaccessible. Text is legible without horizontal scrolling.
Severity:	Medium
Passed?:	Yes

Table 136: Test Case TN-19

5.2.20 Non-Functional TN-20 : Core flows function correctly on mobile viewport

Test ID:	**TN-20**
Category:	Non-Functional
Sub-Category:	Compatibility
Objective:	Verify that the interface works correctly on a mobile viewport width.
Procedure:	1. Open the platform on a device or emulator at 390px width. 2. Complete registration, profile viewing, opportunity search, and document upload. 3. Note any layout breakages or unusable controls.
Outcome:	All core flows complete successfully on the mobile viewport. No controls are obscured or inaccessible. Text is legible without horizontal scrolling.
Severity:	Medium
Passed?:	Yes

Table 137: Test Case TN-20

5.2.21 Non-Functional TN-21 : University adoption suitability and marketability

Test ID:	**TN-21**
Category:	Non-Functional
Sub-Category:	Marketability
Objective:	Verify that the platform appears suitable for university adoption through institution-specific features.
Procedure:	1. Log in as University Staff. 2. Navigate through university-related functions such as restricted postings, events, and aggregated statistics. 3. Observe whether the system communicates a clear reason for a university career centre to use the platform.
Outcome:	The platform shows clear institutional value through university-specific functions. University staff can see support for campus-specific opportunities, events, and student-level insight.
Severity:	Medium
Passed?:	Yes

Table 138: Test Case TN-21

5.2.22 Non-Functional TN-22 : Feedback service failure recovery and user messaging

Test ID:	**TN-22**
Category:	Non-Functional
Sub-Category:	Integration
Objective:	Verify system behaviour when the feedback generation service fails after a document upload.
Procedure:	1. Navigate to the Document Feedback page. 2. Upload a valid document file. 3. Simulate or trigger a failure in the feedback generation service (e.g. API timeout). 4. Observe the document status in the Your Documents section.
Outcome:	The system should not remain indefinitely in a loading state. The document status should update to indicate failure or retry, and the user should receive a clear message that feedback generation could not be completed.
Severity:	High
Passed?:	Yes

Table 139: Test Case TN-22

5.2.23 Non-Functional TN-23 : LLM provider swap requires only configuration change

Test ID:	**TN-23**
Category:	Non-Functional
Sub-Category:	Integration
Objective:	Verify that switching the LLM provider is a configuration-only change requiring no code modification.
Procedure:	1. Identify the environment variable or configuration setting controlling the LLM provider. 2. Change the provider selection to a secondary provider. 3. Run the document feedback and auto-apply workflows. 4. Verify the workflows are complete correctly.
Outcome:	Both workflows complete successfully with the alternative provider. No code changes are required. Provider-specific quirks are handled entirely within the LLM adapter layer.
Severity:	Medium
Passed?:	Yes

Table 140: Test Case TN-23

5.2.24 Non-Functional TN-24 : Extendibility for adding a new opportunity type

Test ID:	**TN-34**
Category:	Non-Functional
Sub-Category:	Extendibility
Objective:	Verify that a new opportunity type can be added without major changes to existing opportunity workflows.
Procedure:	1. Add a new opportunity type such as Fellowship using the existing opportunity model. 2. Create one record of this new type through the recruiter interface. 3. Open the student opportunity listing and detail page. 4. Observe whether the new type appears in the same listing and view structure as current opportunity types.
Outcome:	The system supports the new opportunity type with minimal code changes. Existing job, program, and scholarship flows continue working correctly. The new type appears in the shared opportunity workflow without breaking the interface.
Severity:	Medium
Passed?:	Yes

Table 141: Test Case TN-24

5.2.25 Non-Functional TN-25 : New AI workflow checkpoint registered without changing polling code

Test ID:	**TN-25**
Category:	Non-Functional
Sub-Category:	Extendibility
Objective:	Verify that a new checkpoint type can be added to an AI workflow by registering a new review component without modifying the status polling logic.
Procedure:	1. Add a new checkpoint state to the document feedback or auto-apply graph. 2. Register a new review component for the new checkpoint state. 3. Run the workflow and observe how the frontend handles the new checkpoint.
Outcome:	The polling hook correctly identifies the new checkpoint state and routes to the registered review component. The existing polling code requires no modification. Other checkpoint types continue to function correctly.
Severity:	Medium
Passed?:	Yes

Table 142: Test Case TN-25

5.2.26 Non-Functional TN-26 : Maintainability through diagnostic backend logs

Test ID:	**TN-26**
Category:	Non-Functional
Sub-Category:	Maintainability
Objective:	Verify that backend service logs provide enough information to diagnose a failed workflow.
Procedure:	1. Trigger a controlled failure in a backend workflow such as recommendation refresh or document feedback generation. 2. Open the backend log output or monitoring view. 3. Inspect the recorded messages for request path, failure point, and error context.
Outcome:	The system records meaningful logs that help developers identify where the failure occurred. The logged information is clear enough to support debugging without manually reproducing every internal step.
Severity:	Medium
Passed?:	Yes

Table 143: Test Case TN-26

5.2.27 Non-Functional TN-27 : Prompt files treated as production code require evaluation before merge

Test ID:	**TN-27**
Category:	Non-Functional
Sub-Category:	Maintainability
Objective:	Verify that changes to synthesis or evaluator prompts require a passing run on the evaluation fixture set before they can be merged.
Procedure:	1. Modify a prompt file in the document feedback workflow. 2. Attempt to merge the change without running the evaluation set. 3. Run the evaluation fixture set and confirm it passes. 4. Merge the change after the evaluation passes.
Outcome:	The CI pipeline blocks the merge if the evaluation fixture set has not been run or has failed. The merge succeeds only after the evaluation set passes. No prompt change is deployed to production without this gate.
Severity:	Medium
Passed?:	Yes

Table 144: Test Case TN-27

5.2.28 Non-Functional TN-28 : Backend organised into named Django apps with clear responsibilities

Test ID:	**TN-28**
Category:	Non-Functional
Sub-Category:	Maintainability
Objective:	Verify that the backend codebase is organised into named Django apps whose responsibilities correspond to the documented subsystem decomposition.
Procedure:	1. Inspect the Django project directory structure. 2. Identify the apps present and their primary modules. 3. Map each app to the corresponding subsystem in the architecture documentation.
Outcome:	Each documented subsystem has a corresponding named Django app. No app contains code that clearly belongs to another subsystem. Cross-cutting utilities reside in a shared common app accessible by all others.
Severity:	Low
Passed?:	Yes

Table 145: Test Case TN-28

5.2.29 Non-Functional TN-29 : Health, readiness, and operational metrics endpoints available

Test ID:	**TN-29**
Category:	Non-Functional
Sub-Category:	Observability
Objective:	Verify that the system exposes health, readiness, and operational metrics endpoints.
Procedure:	1. Send a GET request to the health endpoint. 2. Send a GET request to the readiness endpoint. 3. Send a GET request to the metrics endpoint. 4. Inspect each response for expected fields.
Outcome:	The health endpoint returns 200 indicating the service is alive. The readiness endpoint confirms the database connection is available. The metrics endpoint returns operational counters useful for monitoring. All three respond within 1 second.
Severity:	Medium
Passed?:	Yes

Table 146: Test Case TN-29

5.2.30 Non-Functional TN-30 : Public API requests logged with request identifiers for debugging

Test ID:	**TN-30**
Category:	Non-Functional
Sub-Category:	Observability
Objective:	Verify that every public API request is logged with a unique request identifier that can be used to trace the request through the system.
Procedure:	1. Send several API requests and note the request identifier returned in the response header. 2. Search the backend logs for each identifier. 3. Verify the log entries contain the endpoint, HTTP method, status code, and elapsed time.
Outcome:	Each API response includes a unique request identifier header. The backend log contains an entry for each request keyed by that identifier. The log entry includes endpoint, method, response status, and duration.
Severity:	Medium
Passed?:	Yes

Table 147: Test Case TN-30

5.3 Justification of Tests

The test cases in this report were not written after the system was built. They were written alongside the implementation, and in many cases before it, as a way of defining what correct behaviour actually meant for each feature. This made the tests function as constraints rather than just verification steps. When a developer started work on a new subsystem, the relevant test cases were already there to answer the question of what done meant. This was particularly important for the more complex parts of the system such as the auto-apply workflow and the document feedback pipeline, where the acceptable behaviour was not obvious from the requirements alone and needed to be spelled out explicitly before any code was written.

The selection of test cases was driven by two priorities. The first was coverage of the flows that mattered most to users. Registration, login, profile management, document upload, feedback generation, and auto-apply are the core paths a student takes through the platform, and any failure in these paths would be immediately visible and damaging. Every one of these flows has dedicated test cases that verify not just the happy path but also the failure cases, the edge cases, and the security boundaries. The second priority was coverage of the contracts between subsystems. The agentic package, the backend, and the frontend communicate through well defined interfaces, and a mismatch at any of these boundaries can cause silent failures that are hard to trace. Tests such as the ones verifying that the profile snapshot schema matches what the graph expects, that the consent token cannot be reused, and that the session state persists correctly through a worker restart were written specifically to lock in these contracts and make regressions immediately visible.

The non-functional test cases were selected to cover the quality attributes the system committed to in its requirements. Performance thresholds for the dashboard, search, and authentication endpoints were turned into explicit test cases so that they could be verified rather than assumed. Security properties such as CSRF enforcement, signed URL access control, and memory-hard password storage were each given a dedicated test case because security requirements are easy to accidentally break during refactoring if they are not continuously verified. Maintainability and extendibility tests were included because the platform is intended to keep running and growing after the course concludes, and the team wanted to make sure that the architectural decisions enabling that growth, such as the configuration-driven LLM provider swap and the single-table opportunity model, were verified as working rather than just described.

Making sure all selected tests passed required a combination of unit testing, integration testing, and manual verification. The backend test suite used Django's built-in test framework alongside pytest to verify individual endpoint behaviour, serializer correctness, and database constraint enforcement. The agentic package used pytest with pytest-asyncio to verify graph behaviour, node outputs, and interrupt handling in isolation using fixture data rather than live API calls. Every node in the document

feedback and auto-apply graphs has a deterministic fallback path that produces sensible output without calling any external service, and this property was essential to making the agentic tests reliable and fast. The frontend flows and the end-to-end scenarios that span multiple subsystems were verified through manual testing against the running system, with the test case procedure steps used directly as a script.

When a test case failed during development it was treated as a signal that either the implementation was wrong or the test case itself had uncovered an ambiguity in the requirements that needed resolving. Both outcomes were considered useful. Several test cases led to design changes rather than just bug fixes. The test verifying that the consent token cannot be reused, for example, led to the introduction of a token consumption record in the database that was not part of the original design. The test verifying that a transient LLM failure does not corrupt the session led to an explicit status transition model for the ApplicationSession that replaced an earlier approach that left the session in an ambiguous state on failure. In this way the tests shaped the system rather than simply reflecting it.

The categorisation of test cases by sub-category was a deliberate organisational choice rather than a cosmetic one. Grouping tests by functional area made it easier to identify gaps during development. If a sub-category had only one or two test cases it was a signal that the team had not thought carefully enough about the failure modes in that area, and additional cases were added as a result. The sub-category structure also made it easier to assign verification responsibility across team members. Each person leading a subsystem owned the test cases in the corresponding sub-category, reviewed them for completeness, and was responsible for making sure they passed before that subsystem was considered done. This created a direct connection between the leadership structure of the project and the verification structure of the test suite.

The severity ratings assigned to each test case were used as a triage tool during the final weeks of the project. Critical and High severity cases were treated as hard blockers. No subsystem was considered ready for the final demonstration if any of its Critical or High severity test cases were failing or had not been executed. Medium severity cases were required to pass before the final report submission. Low severity cases were verified but treated as improvements rather than blockers, and any that revealed minor issues were logged for post-course follow-up rather than holding up delivery. This severity based approach meant the team always had a clear and shared understanding of which problems needed to be solved immediately and which could wait, which reduced the pressure of the final integration period and kept the demonstration preparation on schedule.

6 Other Analysis Elements

From the standpoint of a team that developed the initial version of UppGrad and brought it to a state ready for production use, we considered different maintenance options for the platform. Since UppGrad combines a conventional web application with an agentic AI subsystem and a pair of web scrapers that depend on external sites, we found that a single uniform maintenance plan would not match the actual shape of the work. We analysed maintenance in three different contexts which are maintenance of the AI components, maintenance of the application, and maintenance of the data ingestion pipeline.

6.1 Maintenance of the AI Components

The agentic AI subsystem in the platform is maintained from two perspectives. The first factor is the rapid development of language model providers and the frameworks used to orchestrate them. The second factor is the changing capability of the language models themselves.

For the framework side, the LangChain and LangGraph projects used to build the document feedback and auto apply workflows evolve quickly. During development, LangGraph released breaking changes and revised the recommended way to express graph state, conditional edges, and human in the loop interrupts. In the completed platform, this risk is handled through a defined maintenance process that keeps the AI workflows available during production use. Each minor release of LangChain and LangGraph is followed through an automated pull request bot, and a small fixture of representative graph runs is verified before any version bump is merged. Major version updates are reviewed manually with a written changelog summarising the implications. Keeping the framework current prevents deprecated properties in old versions from forcing emergency rewrites later.

For the model side, the situation is similar but more frequent. The OpenAI and Gemini providers used by the system release new model versions regularly, and new models often provide better quality or lower cost for the same task. Each major model release from the chosen providers is evaluated against a fixed evaluation set of fixture documents and fixture opportunities. The evaluation compares the quality of proposed edits and auto apply packages produced by the new model against the current production model. When the new model performs at least as well at lower cost, the configured model name is updated in deployment through a settings change rather than a code change. When it performs worse on a specific feature, that feature remains on the older model and only the suitable calls are routed to the new one. This process keeps the AI capability current without locking the platform into models approaching deprecation.

A separate concern in the AI subsystem is prompt drift. The synthesis prompt and evaluator prompt in the document feedback workflow encode hard rules about

grounding and verbatim quoting. A future change to one of those prompts without rerunning the evaluation set weakens the platform's anti hallucination guarantees. For this reason, prompt files are treated as production code, and every change to them requires a passing run on the evaluation set before merge.

6.2 Maintenance of the Application

As described in the implementation section, the application side of the project depends on several external libraries and frameworks. From the backend perspective, Django, Django REST Framework, the database driver, and Celery each follow their own release cadence and deprecation policy. Backend updates are handled through the same automated pull request bot used on the AI side. Security updates are merged automatically after passing the full continuous integration suite, while major version updates receive human review. This procedure keeps the backend current and prevents known security debt from accumulating after upstream patches exist.

From the frontend perspective, the React ecosystem evolves faster than the backend stack. New versions of React, the routing library, and the build tool are released often, and breaking changes in major versions are common. Minor releases are followed automatically, while major version upgrades are scheduled as quarterly maintenance tasks rather than being mixed with feature work. Development experience showed that mixing feature work with major version upgrades creates long debugging sessions over which the library broke a given page.

From the database management perspective, schema migrations live in the repository and apply automatically on deployment. Destructive migrations such as column drops or table drops are gated behind explicit human approval in the pull request. This is a deliberate maintenance decision. An accidental destructive migration during a routine deployment is one of the worst recoverable failure modes the platform produces, and human approval makes accidental triggering highly unlikely. Backups follow a defined retention policy and are verified weekly through an automated restore into a disposable environment. During development, the team confirmed that scheduled backups alone are not enough. Backups that have never been restored are not reliable backups.

For bugs, issues reported by users through the in app help form and the GitHub issue tracker are recorded and fixed on a defined release cadence. Routine releases are cut every two weeks and include all minor fixes merged since the previous release train. Critical fixes such as authentication failures or data exposure are released out of band as soon as a patch is verified.

6.3 Maintenance of the Data Ingestion Pipeline

The third and most fragile maintenance context is the pair of web scrapers that ingest external opportunities from LinkedIn. Web scrapers are the most fragile part of any platform that uses them because they depend on the HTML and JavaScript structure of external sites, which changes without notice. In the completed platform, scraper breakage is treated as an expected event rather than an exceptional one.

Each scraper run publishes a parse yield metric, and a daily freshness check compares the parse yield of each scraper against a rolling baseline. A drop beyond the configured threshold raises a maintenance alert with a linked runbook. The runbook covers inspection of a recent successful run, comparison with the current target HTML, parser update, and smoke testing against a small fixture of representative postings. Detecting breakage automatically instead of waiting for a user to notice a stale feed keeps recovery time short.

Rate limiting and polite crawl delays are built into the base scraper class, so a misconfigured run does not accidentally hammer a target site. When a target blocks access entirely, the scraper falls back to its last successful state, avoids bypassing the block, and raises an alert. New scraper targets follow the base class documentation before integration. The integration surface is small enough that a new source usually takes a day of work rather than a week when the new scraper conforms to the same structure as the existing two.

7 Other Project Elements

7.1 Consideration of Various Factors in Engineering Design

7.1.1 Constraints

7.1.1.1 Implementation Constraints

- UppGrad is delivered as a web application with a React based frontend and a Django based backend.
- The system integrates LLM APIs such as OpenAI and Gemini for AI assisted features, including the chatbot and automation support.
- Data collection relies on web scraping and external sources such as job boards.
- Version control and issue tracking are handled through Git and GitHub.
- Persistent storage uses PostgreSQL for structured application data, while CV files and similar documents are stored as uploaded objects.
- Deployment uses cloud hosting, with the architecture prepared for model hosting and heavier workloads during scale up.

7.1.1.2 Economic Constraints

- Core features remain free for newly graduating students, and monetization is handled carefully so that accessibility for students is preserved.
- Job posting APIs are expensive, so scraping is used where it is allowed and technically stable.
- Free tier cloud resources are used during initial operation, and scaling decisions are made according to actual platform usage.
- Free datasets are used for prototyping and optional fine tuning to reduce training cost.

7.1.1.3 Privacy, Security, and Ethical Constraints

- CVs and user profile data are treated as confidential, and they are not shared with third parties without user consent.
- Authentication follows strong practices, including secure password storage and multifactor authentication support.
- The system protects users from automation mistakes, so agentic auto apply features keep a human in the loop and require user confirmation before submissions.
- Recommendations avoid algorithmic bias based on protected or irrelevant attributes, and the system provides high level explanations for results.
- Chatbot conversations are handled as sensitive content, stored and accessed carefully, and explained to users through a clear privacy policy.
- When analytics or model improvement uses user data, opt in and opt out controls exist.

7.1.1.4 User Experience and Usability Constraints

- The platform stays usable for students who are busy and under pressure, with low friction onboarding, clear profile completion guidance, and simple navigation between opportunities, resumes, and applications.
- AI outputs are presented as assistance rather than unquestionable truth, and the interface encourages user review and edits, especially for parsed CV fields and generated text.
- Recommendations are explainable in plain language, so users understand why an opportunity was suggested and what to improve.

Considerations	Effect level	Effect
Public health	**Low**	Indirect effects through guidance, stress, and decision support.
Public safety	**High**	Sensitive documents, account security, and personal data protection.
Public welfare	**Moderate**	Broader access to structured opportunities and tracking tools. Not applicable to every member of society
Global factors	**Low**	We started the project with the aim of expansion globally, however the running for this step will operate locally.
Cultural factors	**Moderate**	Different norms for documents and communication.
Social factors	**Moderate**	Fairness and bias concerns in ranking and guidance.
Environmental factors	**High**	Compute usage and cloud footprint should be controlled. API tokens used for LLMs consume energy more than any code regulation system might cost.
Economic factors	**High**	LLMs cost a lot, making the operational costs for the system increase.

Table 148: Factors that can affect analysis and design

7.1.2 Standards

For requirements and documentation quality, we aligned our requirements writing and SRS structure with ISO IEC IEEE 29148. This helped us write requirements that are testable, unambiguous, and traceable, instead of mixing design decisions with user needs. It also fit the report structure because it gave a clear separation between functional requirements, quality requirements, constraints, and verification planning.

For software quality, we used the ISO IEC 25010 quality model as a checklist when defining quality requirements and reviewing the system before demo and release. This kept the team focused on reliability, performance efficiency, usability, maintainability, and security, instead of focusing only on features.

For security verification, we used the OWASP Application Security Verification Standard as a baseline for web application controls. This gave the team concrete items to test, including authentication controls, access control, input handling, and secure design checks, instead of relying on assumption. For password storage and session handling details, we followed the OWASP Cheat Sheet Series guidance, since those two areas are common failure points and map directly to the login, signup, and session features.

For accessibility, we aligned the user interface with WCAG 2.2 success criteria, targeting the common A and AA expectations where feasible. This mattered because users interact with dashboards, forms, and document upload flows, and accessibility issues often remain hidden until late stages unless they are checked against a standard early.

For modeling and communication, we used UML for class and relationship modeling and for keeping the core domain consistent across frontend and backend. In practice, the standard was less about formal compliance and more about using UML consistently to reduce confusion during implementation and demo.

7.2 Ethics and Professional Responsibilities

Working on UppGrad meant working on a platform that affects real career outcomes for real people, and we took that seriously throughout the project. This subsection records the ethical and professional responsibilities we recognised, the ones we observed in our day to day work, and the ones we fulfilled through specific design and implementation choices.

Responsibility of honesty in what the AI produces on behalf of users

Career platforms that use language models easily slip into producing CVs that misrepresent the candidate, either by inflating real experience or by inventing skills the user does not have. We recognised early that a platform that fabricates a credential

damages a user's credibility with employers, and the harm in that direction is significant and hard to undo. We fulfilled this responsibility through three concrete mechanisms in the agentic AI subsystem. The synthesis prompt in document feedback enforces a verbatim quote rule, so every proposed edit references text that actually exists in the user's document. The same prompt forbids inventing skills, awards, or experiences not present in the profile. A separate evaluator step runs after synthesis to check the proposals for groundedness and reject anything that fails. These were not optional features, and they were not added at the end as a checkbox. They were the load bearing rules from the start.

Responsibility of honesty about the limits of the AI itself

We recognised that presenting AI output as authoritative is both wrong and dangerous. A user who treats a confident sounding suggestion as final gives up editorial control over their own application, even though the platform exists to support that control. We fulfilled this responsibility by treating every AI output as a proposal subject to user review, with the per proposal checkbox screen in document feedback and the two human checkpoints in auto apply. We also fulfilled it by being honest in the user interface about what the system does and does not do, rather than using marketing language that overstates capability.

Responsibility of caring with personal data

We recognised that students using UppGrad share genuinely sensitive material, including the full list of opportunities they are applying to, draft personal statements that have not been published anywhere, and correspondence with recruiters. We fulfilled this responsibility through technical design choices documented earlier in the report, including opaque identifiers for uploaded files, signed URLs for retrieval, server side ownership re derivation, and profile field stripping before any external API call. We also fulfilled it through process choices observed in day to day work. We did not log message bodies in plain text, we did not include personal data in shared development fixtures, and we did not paste real user material into language model providers during testing.

Responsibility of making an honest representation of what we built

We recognised that it is easy in a senior design context to overclaim, both in reports and in demonstrations, and that the more capable a system sounds, the more likely the demonstration is to disappoint. We fulfilled this responsibility by recording in this report the things that did not work as originally planned, including the deferral of single sign on, the change to the consent model for auto apply, and the boundedness of the scrapers to two specific external sources. A report that reads as a perfect execution story is a report that has not told the whole truth.

Responsibility of professional behaviour toward external services

The scrapers in particular touch sites we do not own. We recognised that politeness toward those sites is not optional. We fulfilled this responsibility through rate limiting and crawl delays built into the base scraper class, through restraint when a target site signalled that it did not want to be scraped, and through refusal to bypass blocks when they appeared. A scraper that acts aggressively gets the platform blocked, and beyond the practical concern, behaving aggressively toward a site that has not consented to being scraped is the wrong thing to do.

The responsibility of fairness in algorithmic ordering

We recognised that an algorithm that orders applicants for recruiters or recommendations for students makes decisions that affect what a person sees and what opportunities reach them. We fulfilled this responsibility by keeping protected characteristics out of the input features for both the recommendation engine and recruiter side ranking, by making the matching weights configurable, and by attaching a short human readable explanation to each recommendation. This lets a student see why something is being suggested instead of receiving an unexplained ordered list. Algorithmic fairness in career matching is a research problem we did not solve, but the choices we made were the correct choices at our scale.

The responsibility towards each other as a team

Senior design projects often become uneven, with one or two members carrying the load while others coast. We recognised that this is unfair and produces worse outcomes. We fulfilled this responsibility through the work package structure described in the project plan, through regular rotation of meeting facilitation and note taking, and through honest conversations when scheduling pressures meant a member needed help on a specific deliverable. The teamwork details that follow describe how this worked in practice.

7.3 Teamwork Details

7.3.1 Contributing and functioning effectively on the team

The team organized the project according to major system responsibilities and report deliverables. The main areas of work included recommendation flow, opportunity matching, application tracking, recruiter side operations, profile management, document feedback, privacy controls, subsystem decomposition, and test case preparation. Each member was responsible for specific parts of the project, while design decisions affecting multiple subsystems were reviewed together.

A considerable amount of team effort was spent on the document feedback mechanism and the agentic AI related parts of the system. These were not small standalone features. They affected profile handling, external service integration, user facing behavior, storage, subsystem boundaries, and testing. Because of this, the team

reviewed these parts repeatedly while preparing the architecture, refining the diagrams, and writing the subsystem descriptions. Discussions focused on how uploaded documents move through the system, how feedback is returned and stored, how profile data supports both recommendation and feedback generation, and how external intelligent services are represented in the design without making the internal flow unclear.

The recommendation flow also required coordinated work across multiple members. This required repeated revision, especially when new design choices were introduced, such as separating profile management from document feedback and making the external AI related part of the system more explicit in the architecture.

To support this work, the team held regular meetings to review progress, revise naming decisions, discuss feature boundaries, and identify inconsistencies between report sections. Shared folders and common working documents were used for reports, architecture diagrams, notes, and references. This made it easier for members to continue each other's work when needed and helped maintain consistency across the report. It also reduced the risk of one section being updated while related sections remained outdated.

The team's workflow was effective because it combined ownership with regular cross checking. Members did not only complete isolated tasks. They also reviewed how their work affected related parts of the project, especially in areas where the feedback mechanism and agentic AI related processing interacted with other subsystems.

7.3.2 Helping creating a collaborative and inclusive environment

The team maintained a working environment in which technical decisions were discussed openly and revised when necessary. This was particularly important in parts of the project where the design was less straightforward, especially the document feedback mechanism and the agentic AI related services. These areas involved questions about system responsibility, user trust, usefulness of generated output, and the extent of external service dependence. As a result, they required input from more than one perspective.

Team discussions included not only implementation and architecture concerns, but also whether the produced feedback was actually useful to students. This was important because a feedback feature easily becomes generic or unhelpful when treated only as an AI output channel. For that reason, the team reviewed this mechanism from several angles, including user clarity, system flow, data handling, and testability. Members contributed by questioning how feedback is displayed, what kinds of suggestions are expected, how the result connects back to profile improvement, and how the system avoids presenting low quality output as reliable.

A similar level of discussion was needed for the agentic AI related parts of the design. The team determined what is handled by the platform itself and what is delegated to external intelligent services. This affected subsystem placement, the role of the external wrapper, and the overall clarity of the architecture. Open discussion helped prevent vague design choices and made the final structure more understandable to all members.

The team also monitored workload and reassigned work when needed. Some parts of the project demanded more time than initially expected, especially subsystem decomposition, diagram refinement, and the preparation of test cases for AI supported and feedback related flows. In such cases, members supported one another by taking over connected writing tasks, checking report consistency, or helping revise diagrams and definitions. This made the workload more manageable and reduced the chance that one difficult section delayed the rest of the report.

The environment was collaborative because important parts of the project were not kept within one person's area only. Members reviewed shared architectural decisions, raised concerns when something was unclear, and contributed to sections outside their own primary responsibility when the project required it.

7.3.3 Taking lead role and sharing leadership on the team

Leadership was shared across the team by giving each member ownership of a specific area while still contributing actively to every other part of the project. No one was siloed into only their lead role, as every member participated in design decisions, cross-checked each other's work, and stepped in wherever the project needed them. That said, there were clear areas of ownership that each person drove forward.

Emre Uçar took the lead in documentation and report preparation. This included organizing sections, keeping terminology consistent across all deliverables, revising subsystem descriptions, and making sure that every architectural update was reflected throughout the report accurately and on time. This also includes bringing up the constraints of the project, taking part in active testing and reporting the inconsistencies so that the team can take action faster.

Begüm Filiz Öz led the backend and middleware side of the project. This included the Django application structure, REST API design, session and authentication handling, database schema decisions, and making sure that the backend correctly served all the workflows the rest of the platform depended on. Beyond the backend, Begüm also contributed to the frontend, reviewing interface behaviour, cross-checking how user actions mapped to API responses, and ensuring that the two sides of the platform stayed consistent with each other.

Koray Sevil led the frontend, user interface, and middleware side of the project, focusing on the React application structure, screen responsibilities, the presentation layer, and the connection between user actions and backend services. This included the AI workflow checkpoint screens, the polling logic, and keeping the interface consistent across both the student and recruiter shells. Koray also contributed to the backend, reviewing endpoint behaviour and making sure the frontend and backend integration points worked correctly end to end.

Ali Özhavala led the agentic AI related part of the project. This covered the LangGraph workflow design for both document feedback and auto-apply, decisions about how external language model providers were integrated, how the graph state and human checkpoints were structured, and how the agentic package connected back into the rest of the platform.

Bertan Turgut led testing and the project demonstrations. This included shaping the functional and non-functional test cases so that the main flows of the system were covered properly, preparing the supervisor demos at each milestone, and making sure the delivered system could be clearly presented and verified end to end.

Although each person had a clear lead area, these responsibilities were closely connected in practice. Changes in one part regularly required updates in diagrams, report sections, or related subsystems, and the team coordinated across all of them throughout the year.

7.3.4 Meeting objectives

The project plan, included in the Analysis and Requirements Report, set out the deliverables for each work package and the overall timeline. We met the great majority of those objectives, and the cases where we did not match the original plan are recorded honestly.

The Research work package delivered both the literature review and the technology stack decisions on schedule. The Frontend and Design work package delivered the wireframes, the implemented React application, and the integrated frontend backend communication layer on schedule. The Backend work package delivered the Django implementation including authentication, profile management, opportunity management, and application tracking on schedule. The Database Regulation work package delivered the schema, the migrations, and the Neon configuration on schedule. The Web Scraping work package delivered both the LinkedIn scraper and the MastersPortal scraper on schedule. The Middleware work package delivered the language model provider wrapper, the privacy helpers, and the audit logging on schedule.

The Auto apply Agentic Workflow work package delivered the workflow design, the orchestration module, and a complete assisted application flow demonstration on

schedule. This was the largest single effort in the project and the one most at risk of slipping, and meeting the schedule on it required deliberate work organisation, including the decision to lift the document feedback graph out of an early auto apply prototype as a separate workflow.

The Testing work package delivered the test plan and a verified system, but the original target of full coverage of every endpoint was relaxed during the year to coverage of the highest risk flows, including authentication, application submission, document feedback, and auto apply. The team budget did not stretch to exhaustive coverage. This was a deliberate scope adjustment recorded in weekly meetings rather than a failure to deliver.

Two original objectives were deferred during implementation. The first was single sign on with Google as a second registration path, deferred in favour of investing the time in the document feedback and auto apply workflows. The second was the student approval step for new employer organisations, gated behind a feature flag rather than required by default for the demonstration deployment. Both deferrals are recorded in the relevant sections of this report.

The Reports and Documentation work package delivered all three required reports on time. The Demos work package prepared the supervisor demonstrations on schedule throughout the year.

Overall, we met the objectives set in the project plan at the level committed to, with two scoped down items and two deferrals recorded openly rather than hidden. The project ran to schedule.

7.4 New Knowledge Acquired and Applied

The project required us to learn a meaningful amount of new material across the stack. The most substantial new technical knowledge was the agentic AI stack, since none of us had built production agentic workflows before. We learned LangChain and LangGraph through the official documentation, the example repositories published by the projects, and deliberate practice through small throwaway prototypes against fixture data before committing to the full production design. The early offline auto apply prototype was the strategy that mattered most here, because it let us understand how state propagation, interrupts, and resumes behave before building load bearing functionality on top of them.

Several members learned Django and Django REST Framework at production depth for the first time. The strategy that worked best was reading the framework source for the parts we did not understand from the documentation alone, which is a skill that pays off long after a single project. The frontend members deepened their knowledge of React, TypeScript, Vite, and Tailwind, with most of the new learning being about how to keep a multi page application internally consistent rather than about any single tool.

The database and middleware leads learned PostgreSQL beyond what coursework had covered, particularly schema design with optional flexibility through JSONB columns and safe migration practices using a managed database service's branching feature. The scraping lead learned how to build resilient web scrapers, including the politeness practices that keep a scraper from being blocked and the parse yield monitoring that detects breakage automatically.

Beyond technical knowledge, all members acquired new knowledge about working as a team on a year long project. We learned to scope deliverables realistically, identify dependencies early, write down decisions rather than relying on memory, and escalate concerns early rather than late. The learning strategy here was retrospective. At each major milestone, we discussed openly what had worked in the previous phase and what had not, then adjusted our practices for the next phase. This is the skill that transfers most directly to professional work after the course concludes.

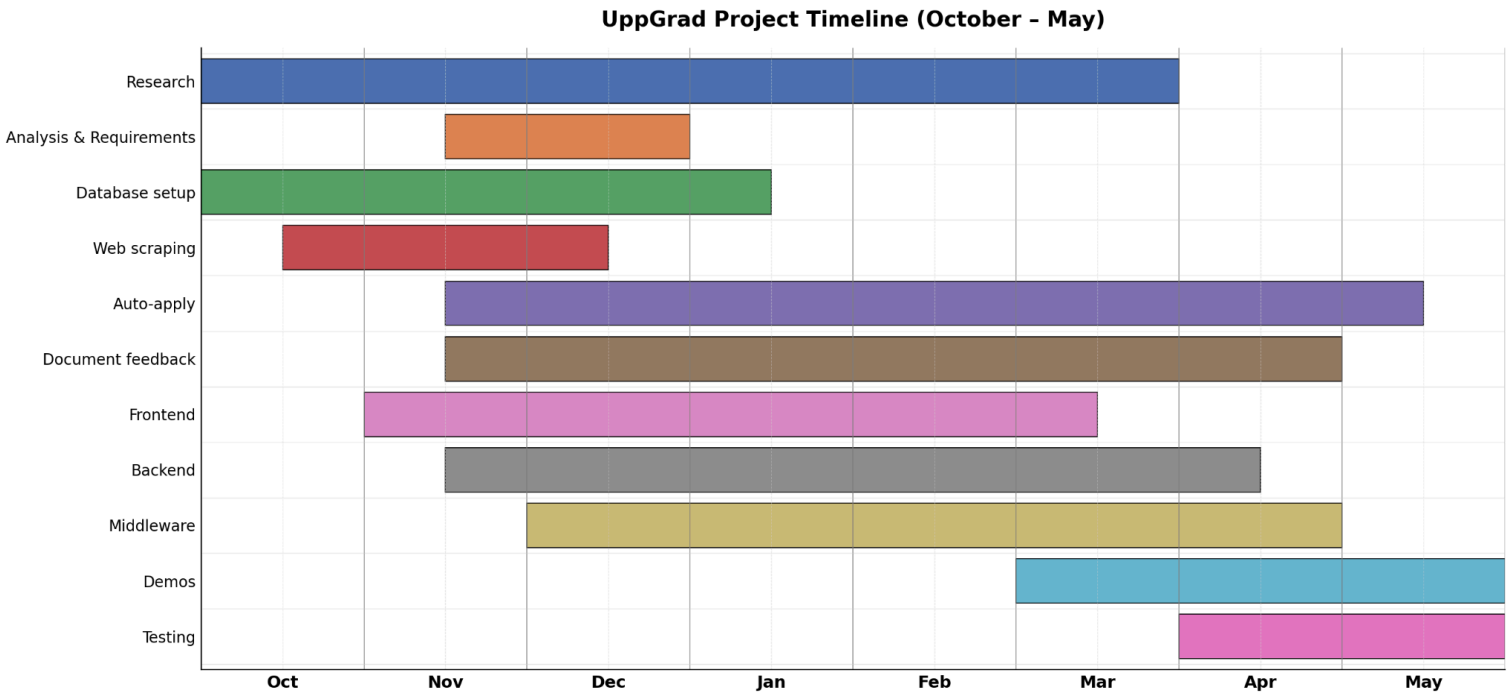

Figure 4: Gantt Chart for the entirety of the Uppgrad project through the entire 2025/2026 academic year

8 Conclusion and Future Work

UppGrad set out to address a fragmented and frustrating problem. University students manage their career searches across many disconnected systems, and the existing tools each solve only part of the problem. The platform we built consolidates those pieces under a single schema, integrates document feedback and auto apply as reviewable human in the loop workflows, and gives recruiters a surface scope to student and early career hiring without the cost of an enterprise system.

The system meets the requirements we committed to and has been delivered in a state ready for production use. The architecture is built to extend rather than to be replaced, and the maintenance plan describes how it will continue to be operated after the course concludes.

The most natural next steps are broader scraper coverage to widen the unified feed, deeper recruiter side analytics, and institutional adoption with single sign-on against university identity providers. Beyond these, the agentic AI subsystem is well placed to support new workflows such as interview preparation grounded in the same profile and opportunity context the existing graphs already use.

The team intends UppGrad to keep running and growing after this report is submitted. The platform works, the path forward is clear, and the responsibility for that path forward is one we are ready to keep carrying.

9 Glossary

AI (Artificial Intelligence): A machine based system that can generate outputs such as predictions, recommendations, content, or decisions that can influence real or virtual environments.

API (Application Programming Interface): A defined interface that lets software components communicate, usually through HTTP requests and responses between client and server.

ASVS (Application Security Verification Standard): A checklist style standard used to verify web application security controls during design, implementation, and testing.

ATS (Applicant Tracking System): Software used by employers to collect, filter, and manage job applications and candidate documents.

Audit Log: A record of security relevant or business relevant actions, such as login attempts, document uploads, consent actions, and application submissions.

Authentication: The process of proving a user is who they claim to be, for example by checking a password, then creating a logged in session.

Authorization: The process of deciding what an authenticated user is allowed to access or do, based on permissions and roles.

Backend: The server side of the system that handles business logic, data storage, and API endpoints.

CDN (Content Delivery Network): A geographically distributed group of servers that caches content closer to users to improve load time and reliability.

CI/CD (Continuous Integration and Continuous Delivery): Automation practices that merge code changes frequently, run builds and tests, and support releasing software in short iterations.

Celery: A Python task queue used to run background jobs such as recommendation refreshes, document feedback, scraping, email sending, and auto apply workflows.

Celery Beat: A scheduler used with Celery to trigger periodic jobs, such as refreshing old recommendation feeds.

Consent Token: A fresh confirmation record showing that the user approved a specific sensitive action, such as an auto apply submission.

Container: A standard unit of software that packages code and dependencies so an application runs consistently across environments.

CRUD (Create Read Update Delete): The four basic operations used when managing stored data through an application.

CSRF (Cross Site Request Forgery): A web attack where an unwanted state changing request is submitted from another site, usually handled through server side CSRF protection.

CV (Curriculum Vitae): A document summarizing education, skills, and experience used for applications.

Database: A system used to store and query persistent data such as users, opportunities, applications, and notifications.

DRF (Django REST Framework): A Django toolkit used to build REST API endpoints between the backend and frontend.

ETL (Extract Transform Load): A pipeline that collects data, reshapes or validates it, then stores it in a target system.

GDPR (General Data Protection Regulation): The EU regulation that sets rules for processing personal data, including user rights and security expectations.

Human in the Loop: A design pattern where the system pauses for user review or approval before continuing a sensitive or uncertain operation.

JWT (JSON Web Token): A compact, URL safe format for representing claims that can be integrity protected or encrypted.

JSONB: A PostgreSQL data type used to store structured JSON data efficiently while still allowing queries over its contents.

LangChain: A framework used to connect language model calls, tools, prompts, and external data sources in AI applications.

LangGraph: A framework used to model AI workflows as graphs with nodes, edges, state, interrupts, and resumable execution.

LLM (Large Language Model): An AI model trained on large amounts of text that can understand and generate language.

MFA (Multi Factor Authentication): An authentication method requiring more than one factor, for example password plus a one time code.

ML (Machine Learning): Techniques that let systems improve performance by learning patterns from training data rather than only explicit instructions.

NoSQL (Not Only SQL): A category of databases that store data in formats other than relational tables, often used for flexible schemas or specific scaling needs.

OAuth 2.0 (OAuth 2.0 Authorization Framework): An authorization framework that lets a third party application obtain limited access to an HTTP service, usually after the user approves access.

Opaque Identifier: A random or non meaningful identifier used to reference uploaded files or records without exposing readable file names or sensitive details.

ORM (Object Relational Mapping): A programming technique that lets application code work with database rows as objects instead of writing raw SQL for every operation.

OWASP (Open Worldwide Application Security Project): A security community that publishes guidance and standards for building and verifying secure web applications.

PostgreSQL: An open source object relational database system commonly used for reliable transactional storage.

RBAC (Role Based Access Control): An access control model where permissions are assigned to roles, and users gain permissions through the roles they have.

REST (Representational State Transfer): An architectural style where clients operate on resources using representations transferred between components, often implemented over HTTP.

Resume: A specific version of a CV that is uploaded and stored so applications can be linked to what was actually submitted.

SMTP (Simple Mail Transfer Protocol): The Internet standard protocol for transporting email, commonly used for account verification messages.

SPA (Single Page Application): A web application that loads one main page and updates content dynamically without full page reloads.

SQL (Structured Query Language): The standard language used to define data structures and operate on data stored in relational databases.

SSO (Single Sign On): An authentication approach that lets users access a system through an existing identity provider, such as a university account.

TLS (Transport Layer Security): A protocol that protects data in transit between client and server against eavesdropping and tampering.

UML (Unified Modeling Language): A standardized visual modeling language used to describe system structure and behavior, for example class diagrams.

UI (User Interface): The screens, forms, and controls a user interacts with.

UX (User Experience): How clear, efficient, and pleasant the product feels while users complete tasks.

WCAG 2.2 (Web Content Accessibility Guidelines 2.2): A W3C recommendation that defines success criteria for making web content more accessible.

XSS (Cross Site Scripting): A web security vulnerability where malicious script is injected into a page and executed in another user's browser.

Object-Oriented Software Engineering, Using UML, Patterns, and Java, 2nd Edition, by Bernd Bruegge and Allen H. Dutoit, Prentice-Hall, 2004, ISBN: 0-13-047110-0.

Krashen, S. (2002). The comprehension hypothesis and its rivals. Selected papers from the Eleventh International Symposium on English Teaching/Fourth Pan Asian Conference. pp. 395-404. Taipei: Crane Publishing Company.

Krashen, S. (2004). Applying the comprehension hypothesis: Some suggestions. Paper presented at 13th international symposium and book fair on language teaching (English Teachers Association of the Republic of China), Taipei, Taiwan. Retrieved from http://www.sdkrashen.com/content/articles/eta_paper.pdf.

Rodrigo, V (2006). The amount of input matters: Incidental acquisition of grammar through listening and reading. The International Journal of Foreign Language Teaching, 2 (1).10-13.

Ponniah, R. J. (2008). Free voluntary reading and the acquisition of grammar by adult ESL students. The International Journal of Foreign Language Teaching, 4 (1) 20-22.

ISO, ISO/IEC/IEEE 29148:2018, Systems and software engineering, Life cycle processes, Requirements engineering, Geneva, Switzerland, 2018.

ISO and IEC, ISO/IEC 25010:2011, Systems and software engineering, Systems and software Quality Requirements and Evaluation, System and software quality models, Geneva, Switzerland, 2011.

OWASP Foundation, OWASP Application Security Verification Standard, Version 5.0.0, May 2025.

OWASP Foundation, Password Storage Cheat Sheet, OWASP Cheat Sheet Series.

OWASP Foundation, OWASP Code Review Guide, Version 2, Jul 2017.

R. T. Fielding, Architectural Styles and the Design of Network Based Software Architectures, PhD dissertation, University of California, Irvine, 2000.

W3C, Web Content Accessibility Guidelines (WCAG) 2.2, W3C Recommendation, 5 Oct 2023.

National Institute of Standards and Technology, Artificial Intelligence Risk Management Framework AI RMF 1.0, NIST AI 100 Part 1, Jan. 2023.

Organisation for Economic Co operation and Development, OECD Framework for the Classification of AI Systems, OECD Artificial Intelligence Papers, 2022.

Internet Engineering Task Force, The OAuth 2.0 Authorization Framework, RFC 6749, Oct. 2012.

Internet Engineering Task Force, JSON Web Token JWT, RFC 7519, May 2015.

Internet Engineering Task Force, The Transport Layer Security TLS Protocol Version 1.3, RFC 8446, Aug. 2018.

Internet Engineering Task Force, Simple Mail Transfer Protocol, RFC 5321, Oct. 2008.

Internet Research Task Force, Argon2 Memory Hard Function for Password Hashing and Proof of Work Applications, RFC 9106, Sep. 2021.

Object Management Group, Unified Modeling Language, Version 2.5.1, Dec 2017.

European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016, Official Journal of the European Union, 2016.

Docker, Inc., What is a container, Docker Documentation.

Cloudflare, What is a content delivery network CDN, Cloudflare Learning Center.

K. Schwaber and J. Sutherland, The Scrum Guide, Nov 2020.

GitHub, Inc., About protected branches, GitHub Docs.

GitHub, Inc., About pull request reviews, GitHub Docs.

Association for Computing Machinery, ACM Code of Ethics and Professional Conduct, 22 Jun 2018.

IEEE, IEEE Code of Ethics, approved Aug 1990.

Django Software Foundation, “Django documentation,” Django Software Foundation, 2026.

Django Software Foundation, “Security in Django,” Django Software Foundation, 2026.

Django Software Foundation, “Cross Site Request Forgery protection,” Django Software Foundation, 2026.

Django Software Foundation, “Password management in Django,” Django Software Foundation, 2026.

Encode OSS Ltd., “Django REST framework,” 2026.

Meta Platforms, Inc., “React documentation,” 2026.

Meta Platforms, Inc., “Built in React Hooks,” React Documentation, 2026.

PostgreSQL Global Development Group, “PostgreSQL 18 documentation,” PostgreSQL Global Development Group, 2026.

PostgreSQL Global Development Group, “JSON functions and operators,” PostgreSQL 18 Documentation, 2026.

PostgreSQL Global Development Group, “Full Text Search,” PostgreSQL 18 Documentation, 2026.

Neon, Inc., “Neon documentation,” Neon Docs, 2026.

Neon, Inc., “Connection pooling,” Neon Docs, 2026.

Celery Project, “Celery Distributed Task Queue documentation,” Celery Documentation, 2026.

Celery Project, “Introduction to Celery,” Celery Documentation, 2026.

LangChain, Inc., “LangGraph overview,” LangChain Documentation, 2026.

LangChain, Inc., “Durable execution,” LangGraph Documentation, 2026.

LangChain, Inc., “Human in the loop,” LangChain Documentation, 2026.

LangChain, Inc., “Agents,” LangChain Documentation, 2026.

LangChain, Inc., “Tools,” LangChain Documentation, 2026.

OpenAI, “OpenAI API platform documentation,” OpenAI Developers, 2026.

OpenAI, “Text generation,” OpenAI API Documentation, 2026.

OpenAI, “Responses overview,” OpenAI API Reference, 2026.

OWASP Foundation, “Cross Site Request Forgery Prevention Cheat Sheet,” OWASP Cheat Sheet Series, 2026.

OWASP Foundation, “File Upload Cheat Sheet,” OWASP Cheat Sheet Series, 2026.